

无人机感知算法

课程内容及答疑:

<https://www.shenlanxueyuan.com/course/385>

目标: 让无人机在未知复杂环境中全自主飞行

周围的环境是什么样子的?

PERCEPTION

我在哪?

SLAM

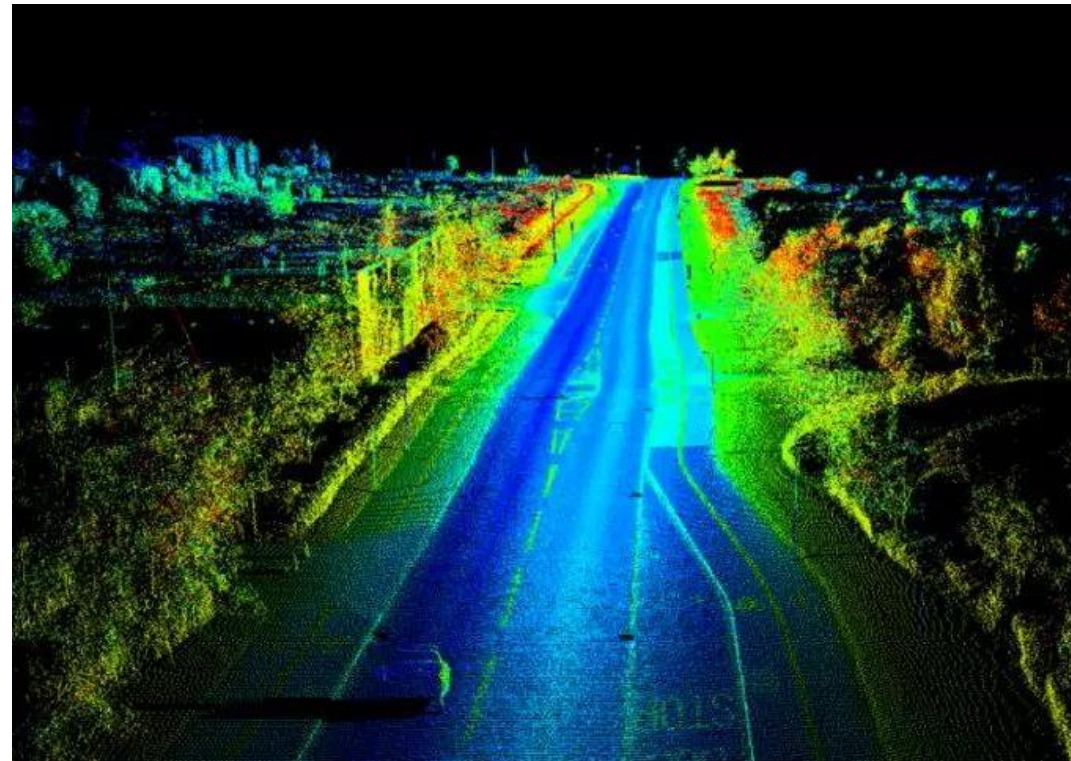
我要去哪?

PLANNING

两项最主要的感知任务



语义感知

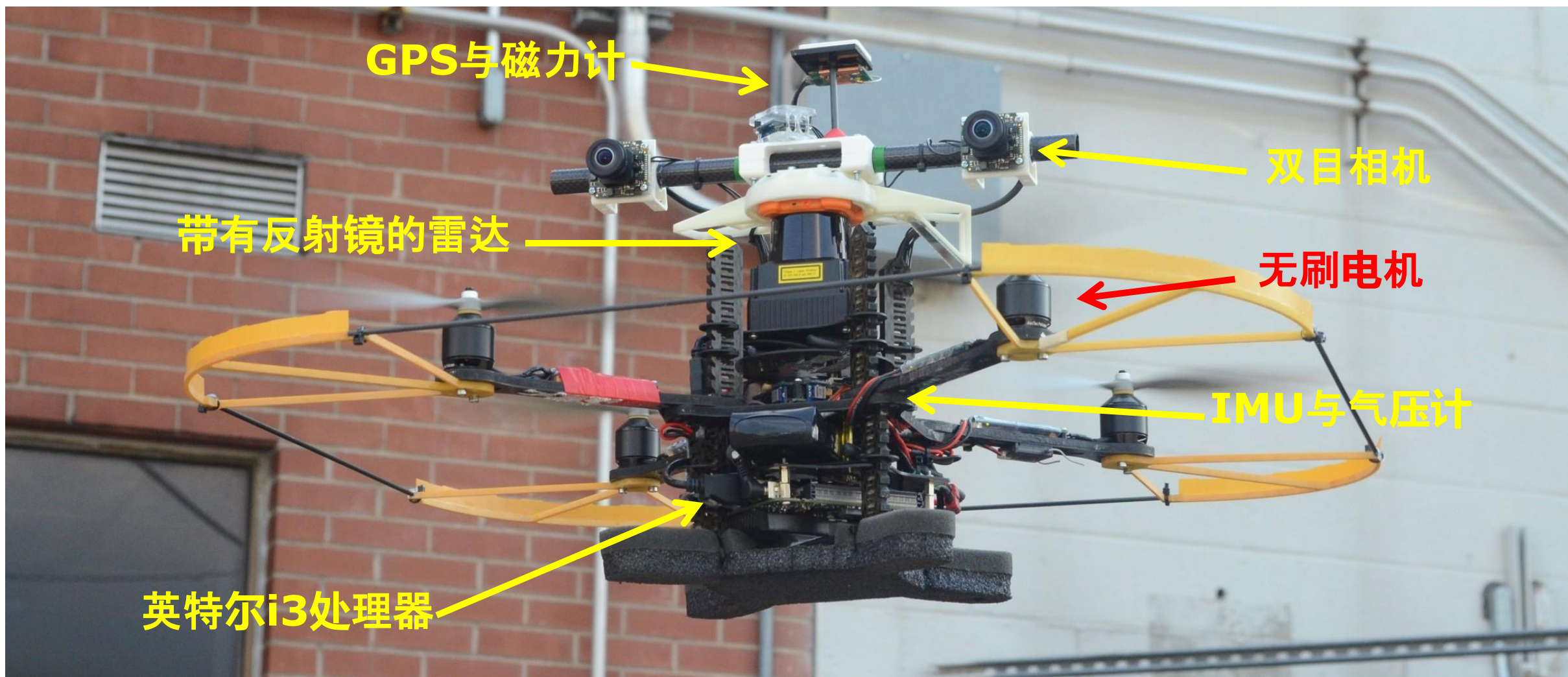


几何感知

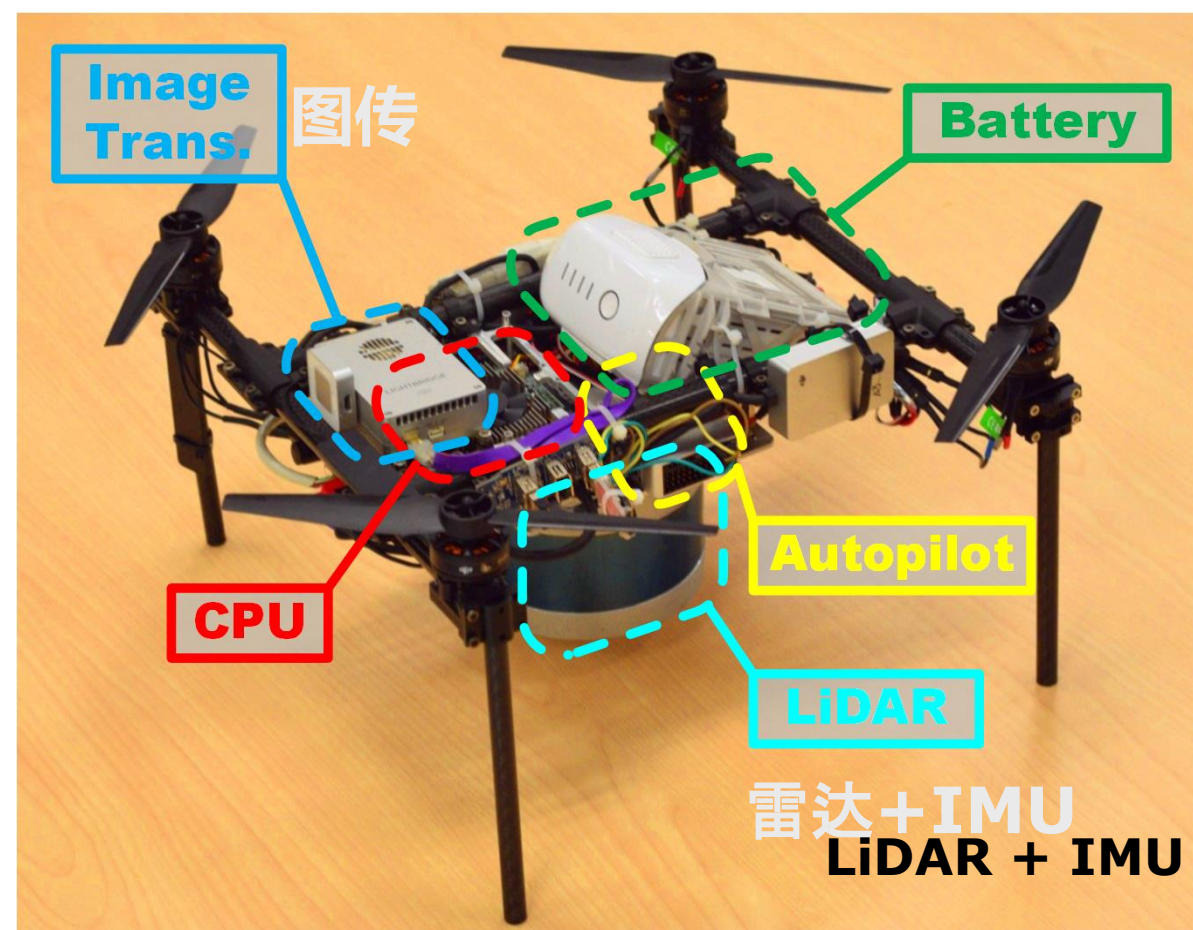
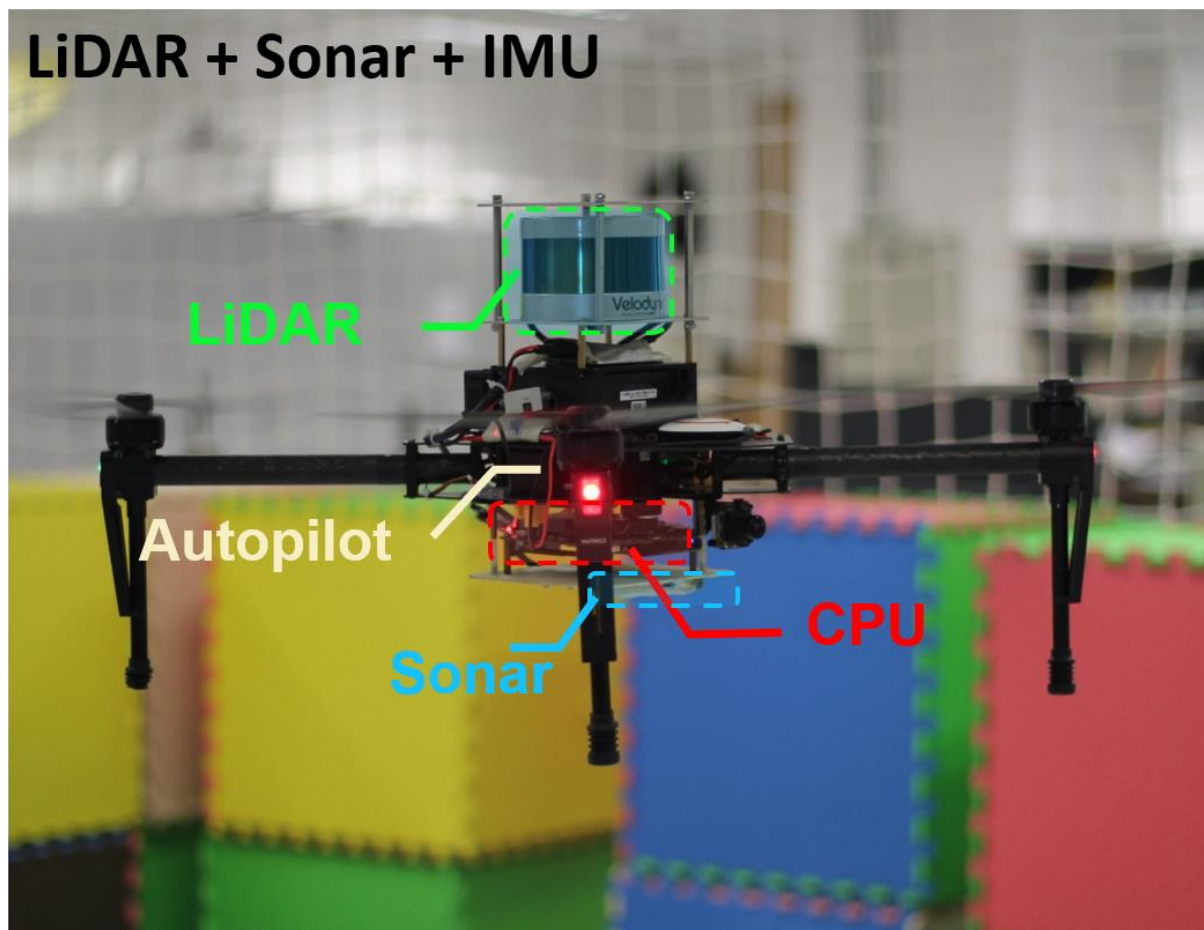
常用传感器



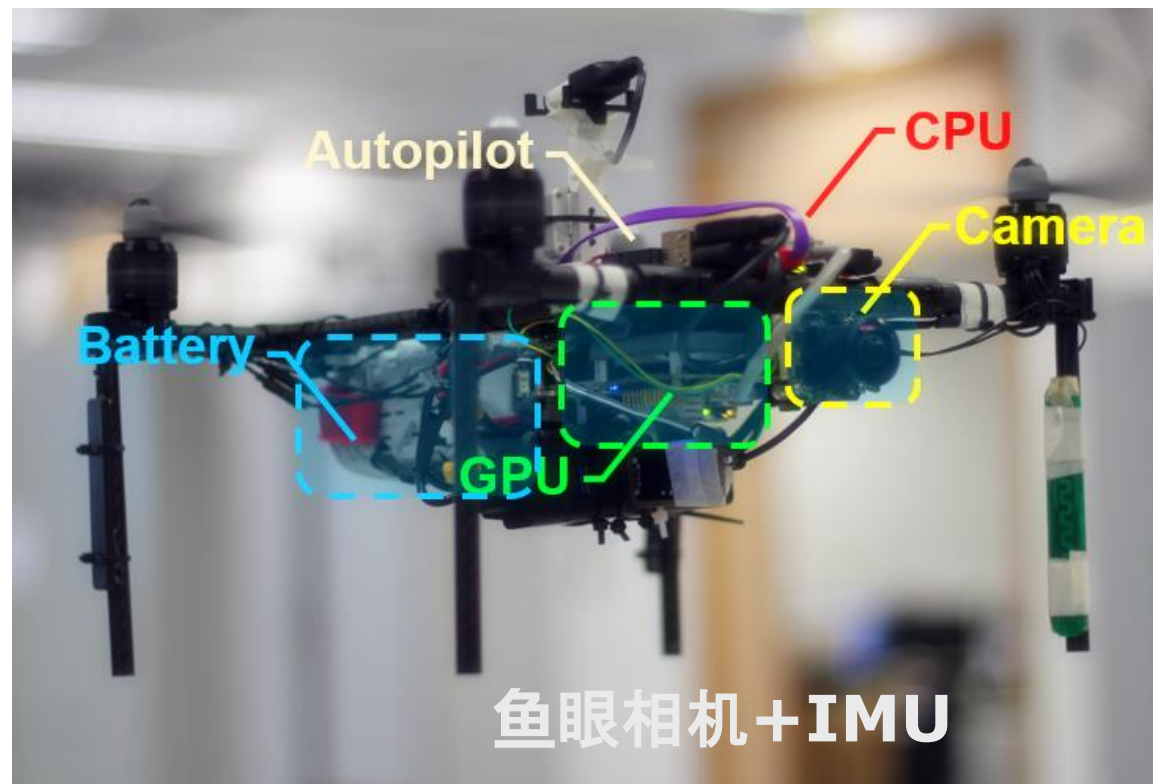
早期自主无人机系统



现在的自主无人机系统

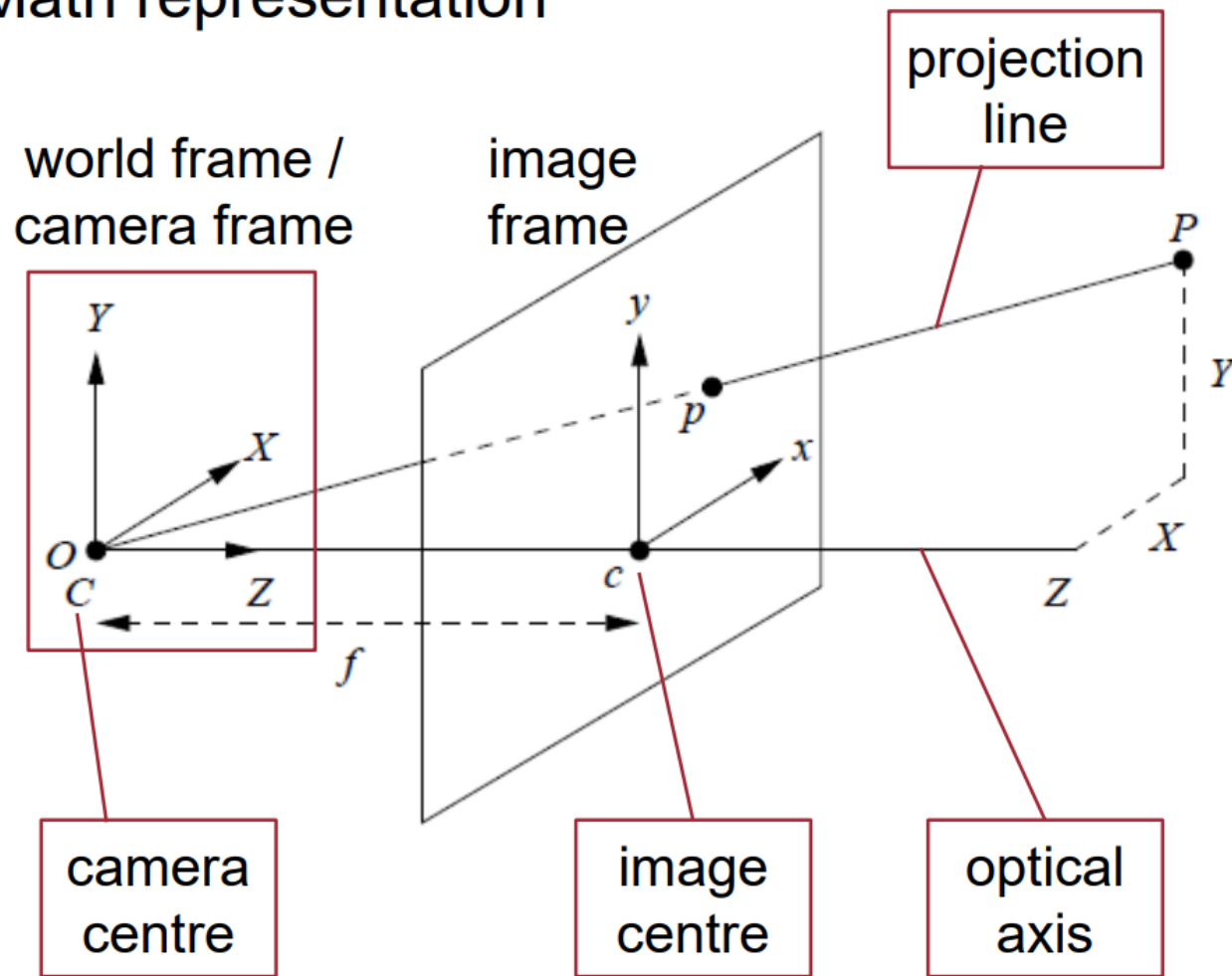


现在的自主无人机系统

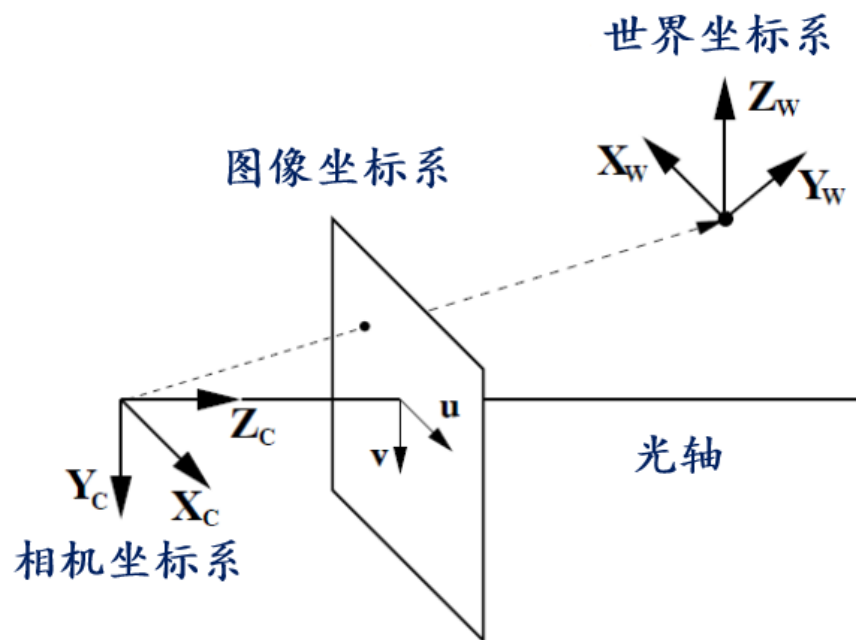


针孔相机模型

Math representation



相机模型



- 光轴：z-轴
- 像平面 (u, v) 垂直于光轴
- 图像中心 (u_0, v_0) ：光轴和像平面交点
- f ：像平面和原点距离（像素）
- 公式：

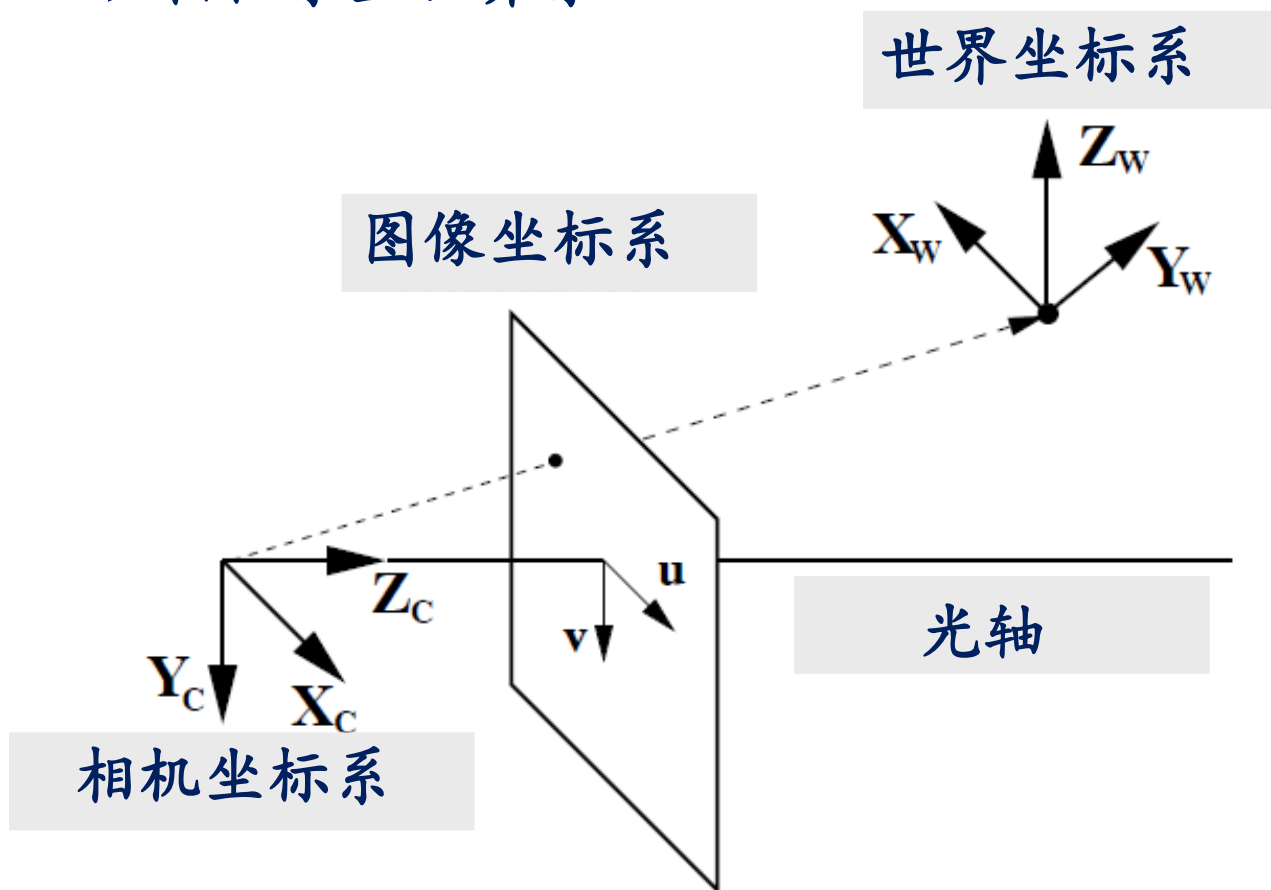
$$u = \frac{fX_c}{Z_c} + u_0$$

$$v = \frac{fY_c}{Z_c} + v_0$$

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f & 0 & u_o \\ 0 & f & v_o \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix}$$

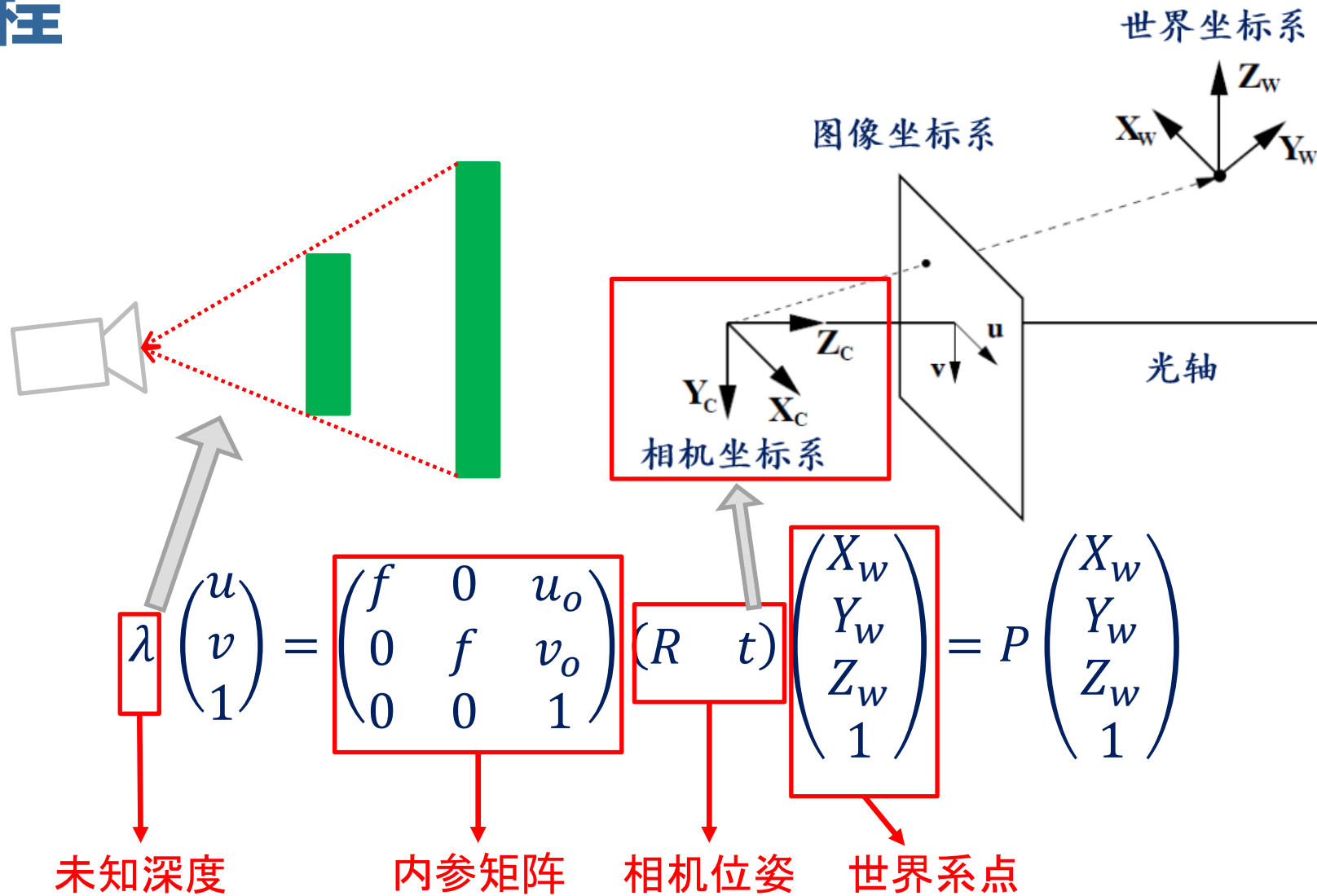
坐标变换

- 从相机系至世界系:



$$\begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{pmatrix}$$

投影方程



RGBD 传感器

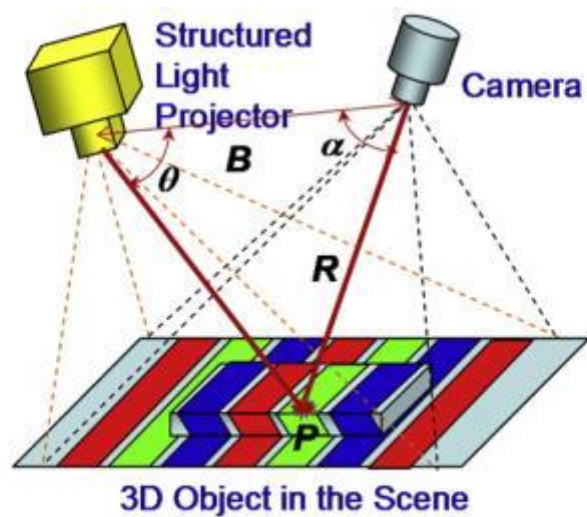
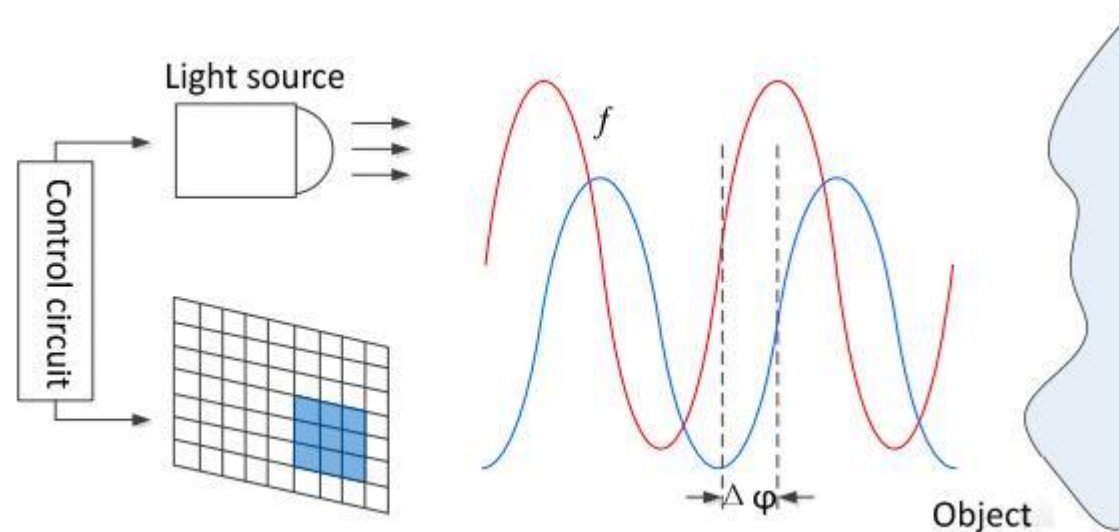


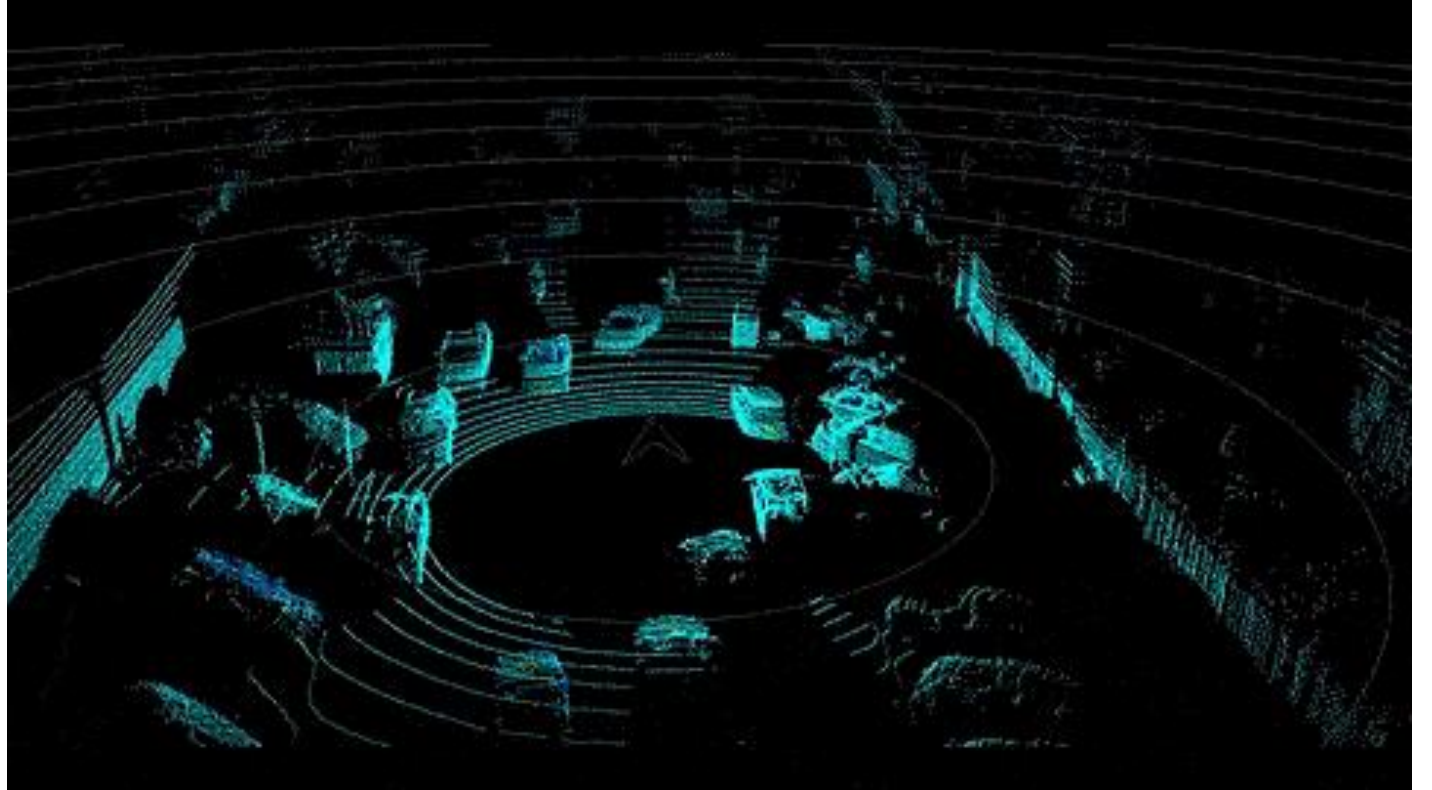
Illustration of structured light.

结构光

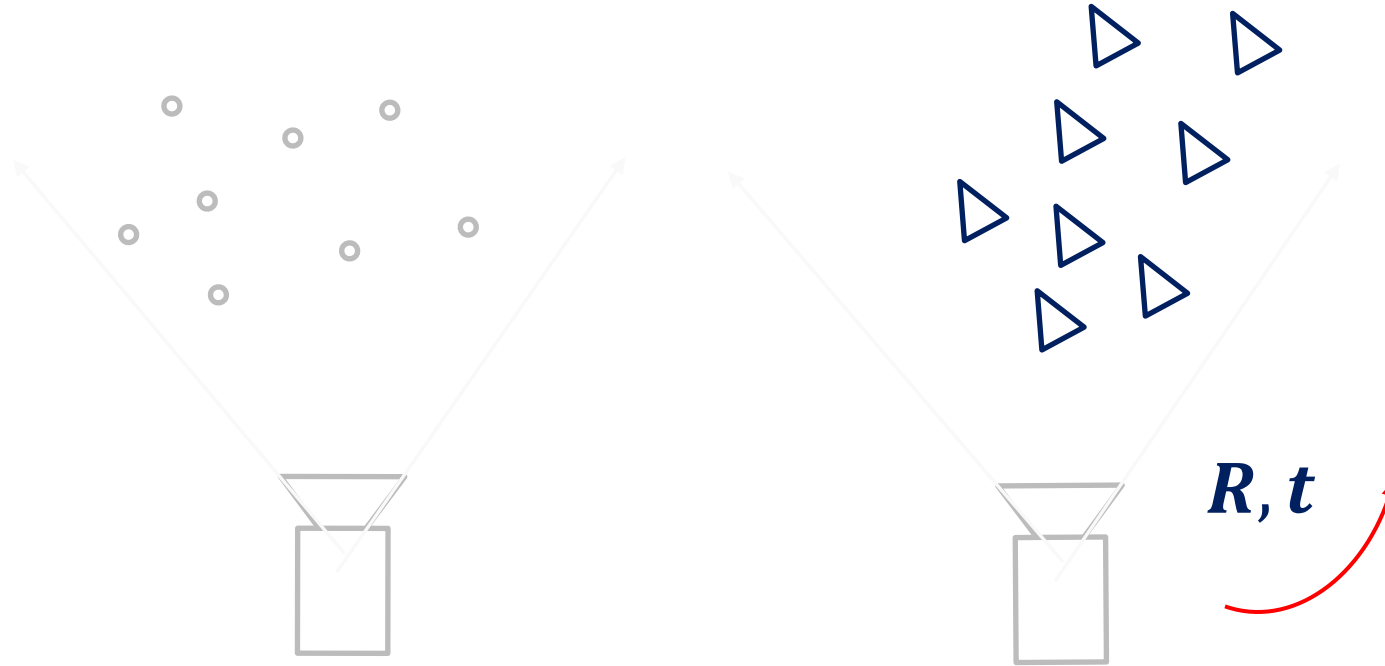


飞行时间(TOF)

激光雷达



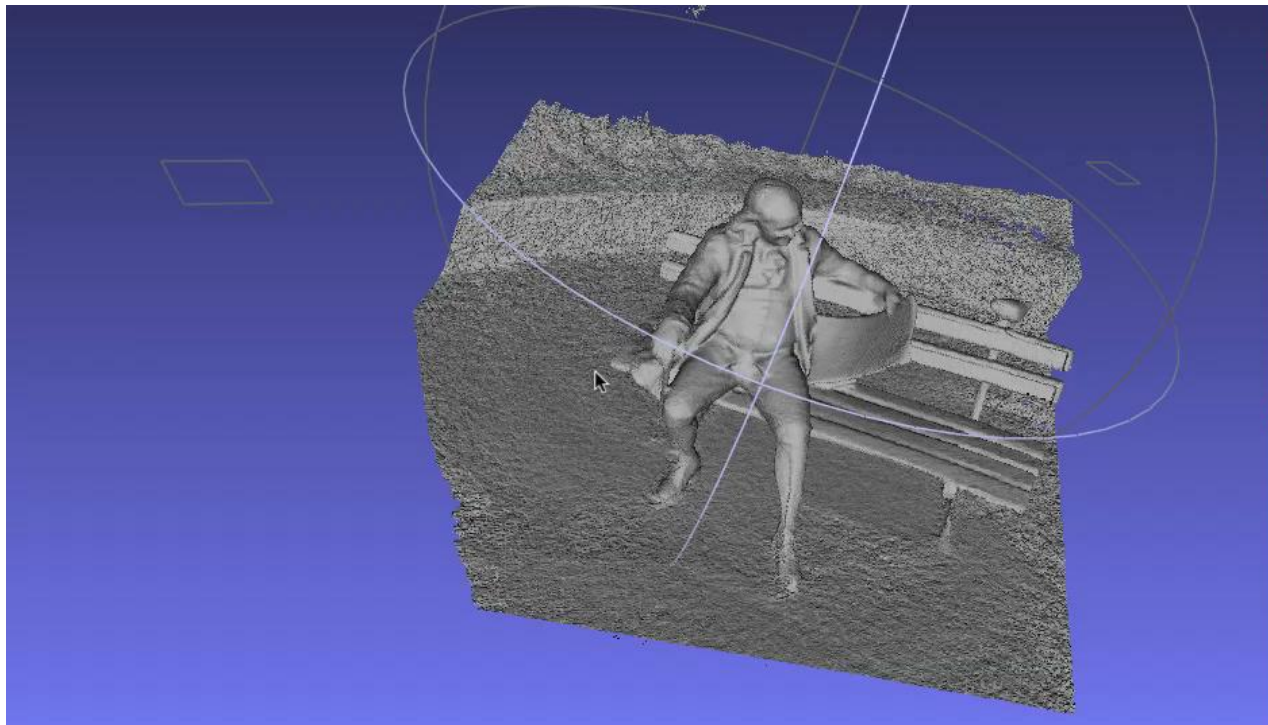
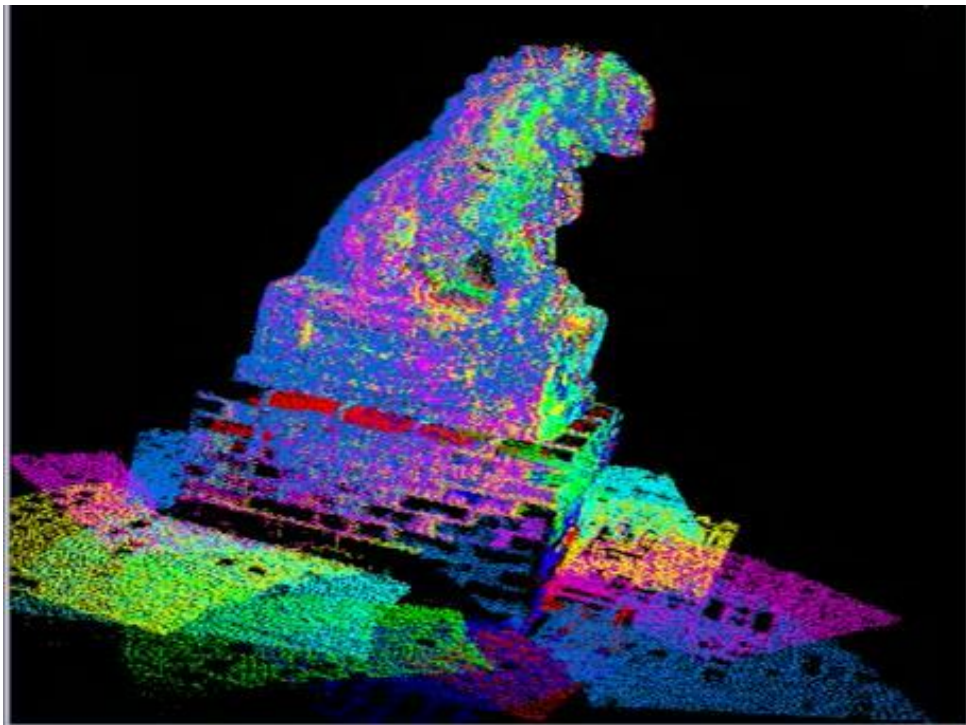
3D-3D位姿估计



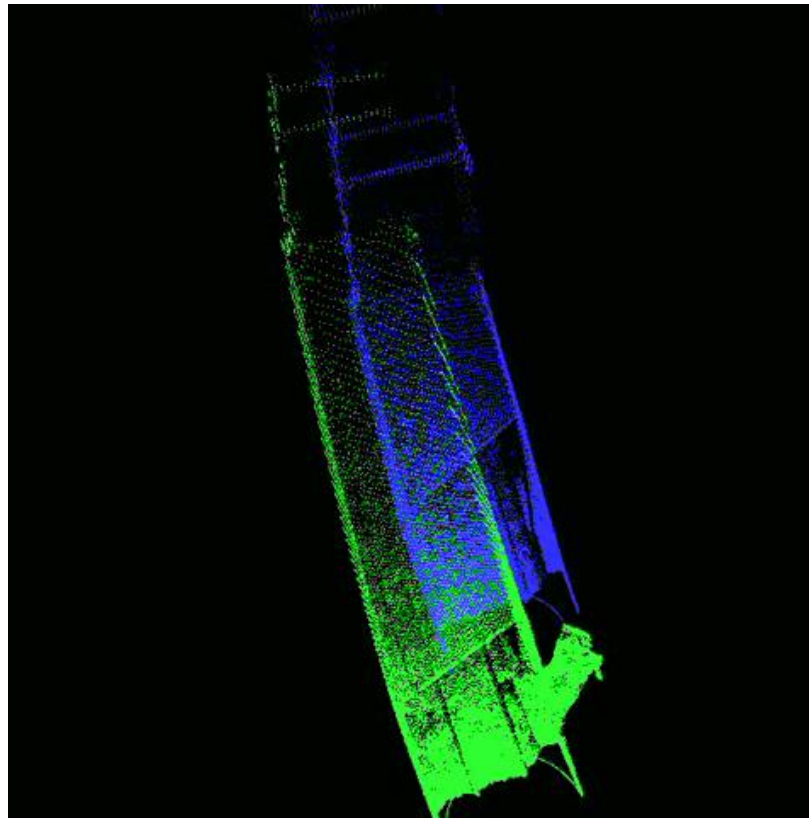
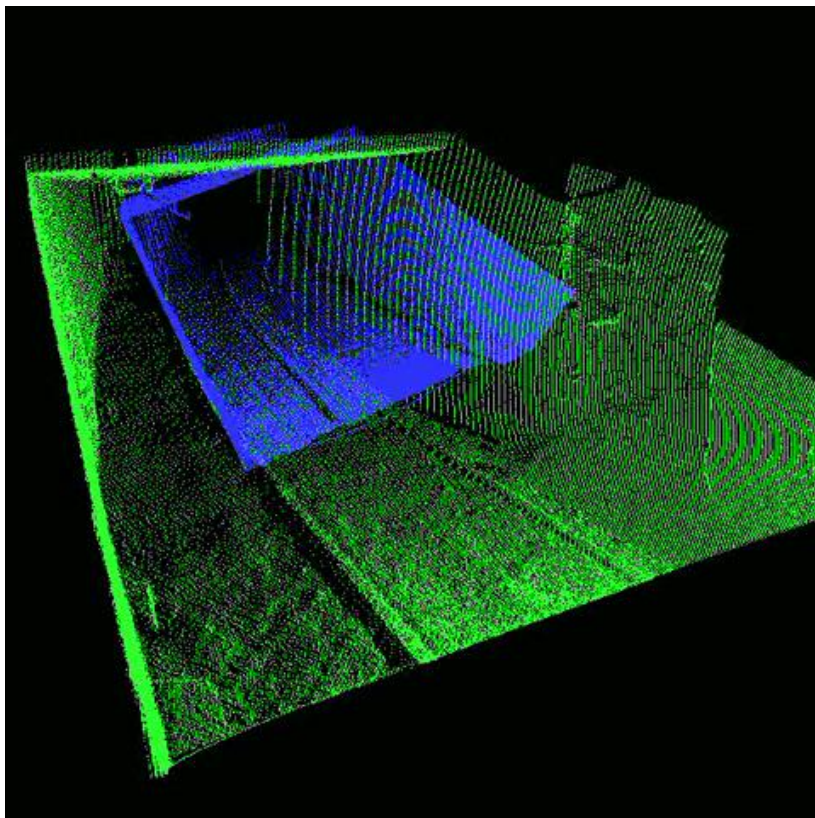
3D-3D

3D-3D位姿估计

- **3D-3D 配准：** 由多点云数据构建统一三维模型

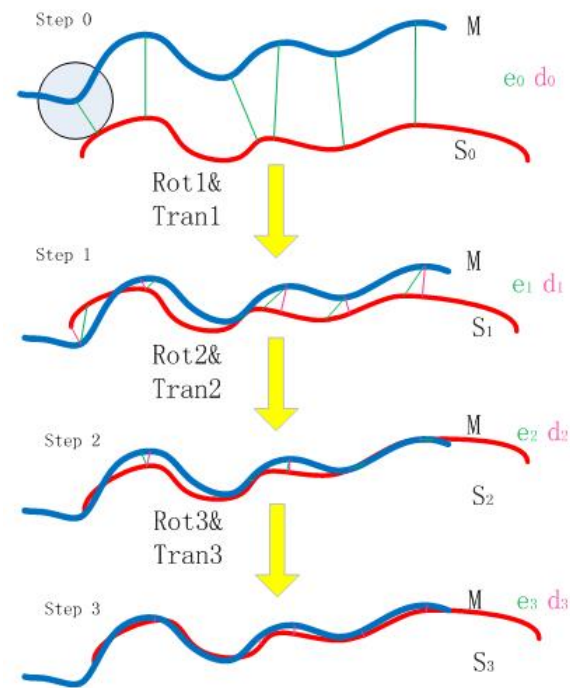


3D-3D位姿估计 (点云配准)



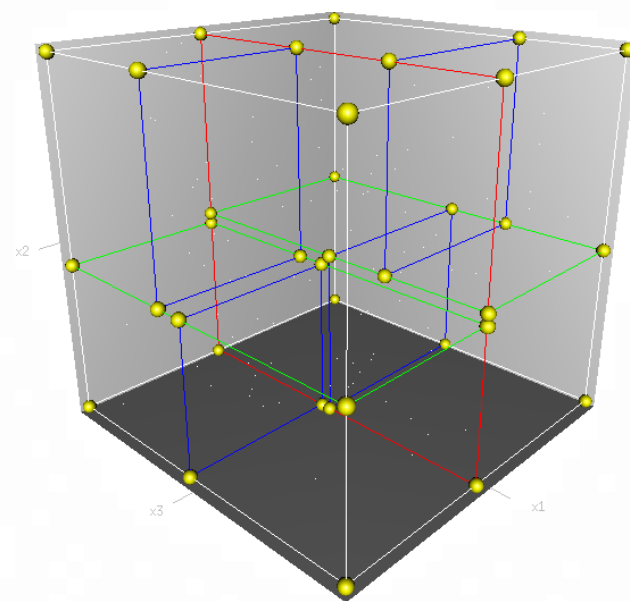
3D-3D位姿估计

- 如何进行数据关联？
 - The Iterative Closest Point (ICP) 算法
 1. 初始化R、T为initial guess
 2. 使用当前RT，对pointcloud1中任意一点，找到它在pointcloud2中的最近点
 3. 使用上一步得到的数据关联，更新RT
 4. 重复2~3直到收敛



3D-3D位姿估计

- 如何进行数据关联?
 - **The Iterative Closest Point (ICP) 算法**
 - 关键步骤: 最近邻查询
 - 线性查找: $O(N)$
 - **K-d Tree: $O(\log N)$**

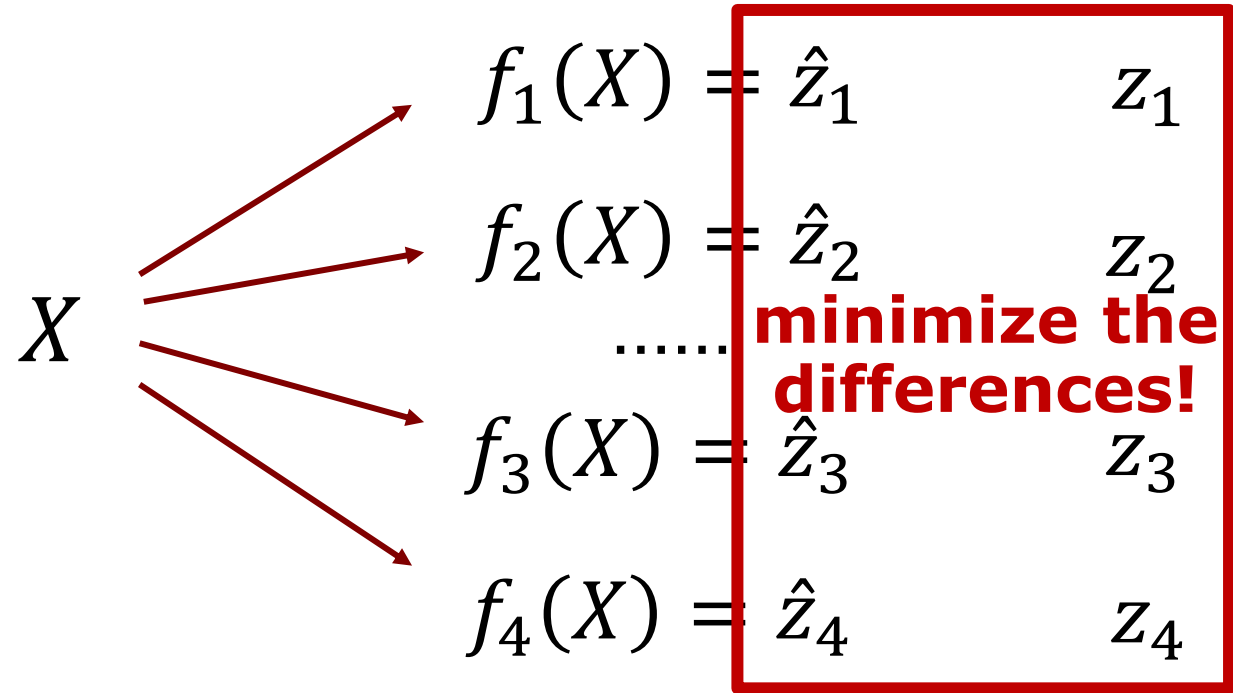


Non-linear Least Squares



Carl Friedrich **Gauss** (1777-1855)

Graphical Explanation



State
(unknown)

Predicted
measurements

Real
measurements

Error Function

- Error $\mathbf{e}_i$ is typically the **difference** between the **predicted and actual** measurement

$$\mathbf{e}_i(\mathbf{x}) = \mathbf{z}_i - f_i(\mathbf{x})$$

- We assume that the measurement error has **zero mean** and is **normally distributed**
- Gaussian error with information matrix $\mathbf{\Omega}_i$
- The squared error of a measurement depends only on the state and is a scalar

$$e_i(\mathbf{x}) = \mathbf{e}_i(\mathbf{x})^T \mathbf{\Omega}_i \mathbf{e}_i(\mathbf{x})$$

Goal: Find the Minimum

- Find the state x^* which minimizes the error given all measurements.

$$x^* = \arg \min_x F(x)$$

global error (scalar)

$$= \arg \min_x \sum_i e_i(x)$$

squared error terms (scalar)

$$= \arg \min_x \sum_i \mathbf{e}_i(x)^T \Omega_i \mathbf{e}_i(x)$$

error terms (vector)

Goal: Find the Minimum

- Find the state $\mathbf{x}^*$ which minimizes the error given all measurements

$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}} \sum_i \mathbf{e}_i^T(\mathbf{x}) \Omega_i \mathbf{e}_i(\mathbf{x})$$

- A general solution is to derive the global error function and find its nulls
 - In general complex and no closed form solution
- ➡ Numerical approaches

Assumption

- A good initial guess is available
- The error functions are “smooth” in the neighborhood of the (hopefully global) minima
- In this case, we can solve the problem by iterative local linearizations

Gauss-Newton: The Overall Error Minimization Procedure

- Define the error function $e_{ij}(\mathbf{x}) = \dots$
- Linearize the error function $e_{ij}(\mathbf{x} + \Delta\mathbf{x}) \simeq \dots$
- Compute its derivative $\mathbf{J}_{ij} = \frac{\partial e_{ij}(\mathbf{x})}{\partial \mathbf{x}}$
- Set the derivative to zero
- Solve linear system $\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$
- Update unknowns $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$
- Iterate

Linearizing the Error Function

- Approximate the error functions around an initial guess $\mathbf{x}$ via Taylor expansion

$$e_i(\mathbf{x} + \Delta\mathbf{x}) \simeq \underbrace{e_i(\mathbf{x})}_{e_i} + \mathbf{J}_i(\mathbf{x})\Delta\mathbf{x}$$

- Reminder: Jacobian

$$\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \frac{\partial f_1(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \frac{\partial f_2(\mathbf{x})}{\partial x_1} & \frac{\partial f_2(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_2(\mathbf{x})}{\partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \frac{\partial f_m(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

Squared Error

- With this linearization, we can fix $\mathbf{x}$ and carry out the minimization in the increments $\Delta\mathbf{x}$
- We replace the Taylor expansion in the squared error terms:

$$\begin{aligned}e_i(\mathbf{x} + \Delta\mathbf{x}) &= \mathbf{e}_i^T(\mathbf{x} + \Delta\mathbf{x})\Omega_i\mathbf{e}_i(\mathbf{x} + \Delta\mathbf{x}) \\&\simeq (\mathbf{e}_i + \mathbf{J}_i\Delta\mathbf{x})^T\Omega_i(\mathbf{e}_i + \mathbf{J}_i\Delta\mathbf{x}) \\&= \mathbf{e}_i^T\Omega_i\mathbf{e}_i + \\&\quad \mathbf{e}_i^T\Omega_i\mathbf{J}_i\Delta\mathbf{x} + \Delta\mathbf{x}^T\mathbf{J}_i^T\Omega_i\mathbf{e}_i + \\&\quad \Delta\mathbf{x}^T\mathbf{J}_i^T\Omega_i\mathbf{J}_i\Delta\mathbf{x}\end{aligned}$$

Squared Error (cont.)

- By grouping similar terms, we obtain:

$$\begin{aligned} e_i(\mathbf{x} + \Delta \mathbf{x}) &\simeq \underbrace{\mathbf{e}_i^T \Omega_i \mathbf{e}_i}_{c_i} + 2 \underbrace{\mathbf{e}_i^T \Omega_i \mathbf{J}_i}_{\mathbf{b}_i^T} \Delta \mathbf{x} + \Delta \mathbf{x}^T \underbrace{\mathbf{J}_i^T \Omega_i \mathbf{J}_i}_{\mathbf{H}_i} \Delta \mathbf{x} \\ &= c_i + 2 \mathbf{b}_i^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H}_i \Delta \mathbf{x} \end{aligned}$$

Global Error

- The global error is the sum of the squared errors terms corresponding to the individual measurements

$$\begin{aligned} F(\mathbf{x} + \Delta \mathbf{x}) &\simeq \sum_i \left(c_i + \mathbf{b}_i^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H}_i \Delta \mathbf{x} \right) \\ &= \underbrace{\sum_i c_i}_c + 2 \underbrace{\left(\sum_i \mathbf{b}_i^T \right)}_{\mathbf{b}^T} \Delta \mathbf{x} + \Delta \mathbf{x}^T \underbrace{\left(\sum_i \mathbf{H}_i \right)}_{\mathbf{H}} \Delta \mathbf{x} \\ &= c + 2\mathbf{b}^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H} \Delta \mathbf{x} \end{aligned}$$

Quadratic Form

- We can write the global error terms as a quadratic form in $\Delta \mathbf{x}$

$$F(\mathbf{x} + \Delta \mathbf{x}) \simeq c + 2\mathbf{b}^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H} \Delta \mathbf{x}$$

- Our goal is to minimize this function
- We need to compute the derivative of $F(\mathbf{x} + \Delta \mathbf{x})$ w.r.t. $\Delta \mathbf{x}$

Deriving a Quadratic Form

- Given a quadratic form

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{H} \mathbf{x} + \mathbf{b}^T \mathbf{x}$$

- its first derivative is

$$\frac{\partial f}{\partial \mathbf{x}} = (\mathbf{H} + \mathbf{H}^T) \mathbf{x} + \mathbf{b}$$

See: The Matrix Cookbook, Section 2.2.4

Quadratic Form

- We can write the linearized global error terms as a quadratic form in $\Delta \mathbf{x}$

$$F(\mathbf{x} + \Delta \mathbf{x}) \simeq c + 2\mathbf{b}^T \Delta \mathbf{x} + \Delta \mathbf{x}^T \mathbf{H} \Delta \mathbf{x}$$

- The derivative of $F(\mathbf{x} + \Delta \mathbf{x})$ w.r.t. $\Delta \mathbf{x}$ is then:

$$\frac{\partial F(\mathbf{x} + \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} \simeq 2\mathbf{b} + 2\mathbf{H} \Delta \mathbf{x}$$

Minimizing the Quadratic Form

- Derivative $\frac{\partial F(\mathbf{x} + \Delta\mathbf{x})}{\partial \Delta\mathbf{x}} \simeq 2\mathbf{b} + 2\mathbf{H}\Delta\mathbf{x}$

- Setting it to zero leads to

$$0 = 2\mathbf{b} + 2\mathbf{H}\Delta\mathbf{x}$$

- Which leads to the linear system

$$\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$$

- The solution for the increment $\Delta\mathbf{x}^*$ is

$$\Delta\mathbf{x}^* = -\mathbf{H}^{-1}\mathbf{b}$$

Overall Procedure

Iterate the following steps:

- Linearize around $\mathbf{x}$ and compute for each measurement

$$\mathbf{e}_i(\mathbf{x} + \Delta\mathbf{x}) \simeq \mathbf{e}_i(\mathbf{x}) + \mathbf{J}_i\Delta\mathbf{x}$$

- Compute the terms for the linear system $\mathbf{b}^T = \sum_i \mathbf{e}_i^T \Omega_i \mathbf{J}_i$ $\mathbf{H} = \sum_i \mathbf{J}_i^T \Omega_i \mathbf{J}_i$

- Solve the linear system

$$\Delta\mathbf{x}^* = -\mathbf{H}^{-1}\mathbf{b}$$

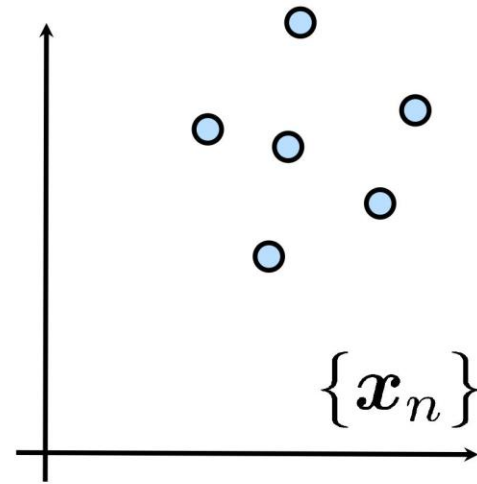
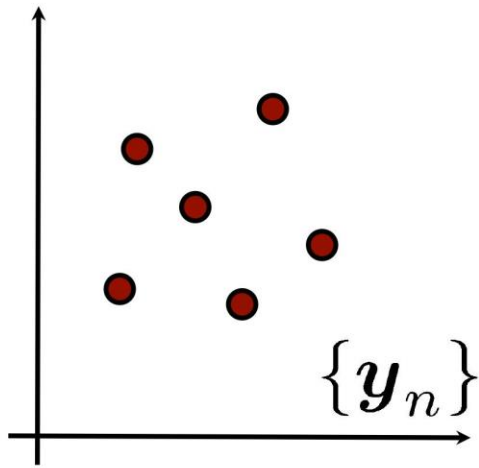
- Updating state $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}^*$

Gauss-Newton: The Overall Error Minimization Procedure

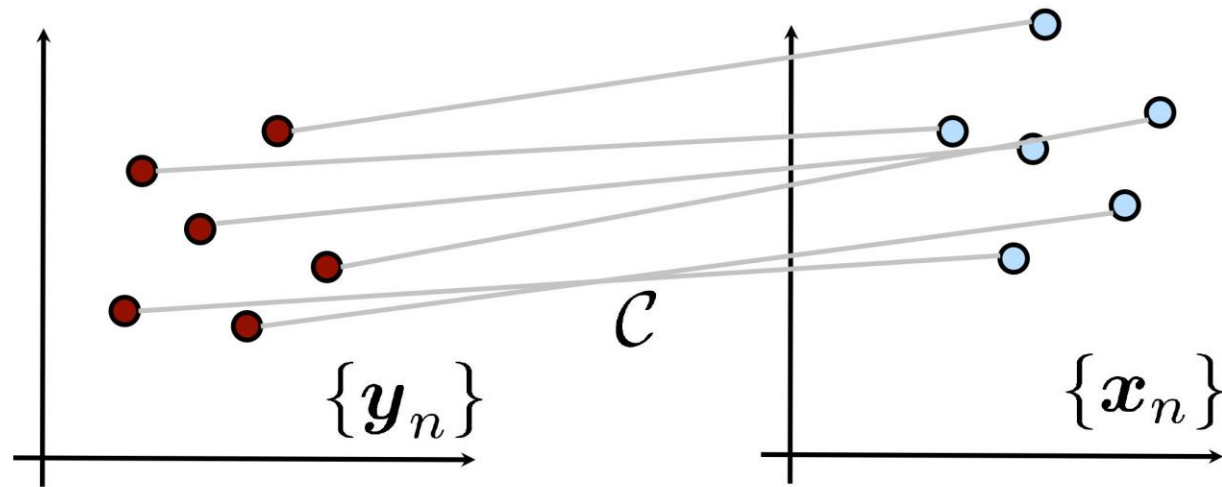
- Define the error function $e_{ij}(\mathbf{x}) = \dots$
- Linearize the error function $e_{ij}(\mathbf{x} + \Delta\mathbf{x}) \simeq \dots$
- Compute its derivative $\mathbf{J}_{ij} = \frac{\partial e_{ij}(\mathbf{x})}{\partial \mathbf{x}}$
- Set the derivative to zero
- Solve linear system $\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$
- Update unknowns $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$
- Iterate

Point Cloud Registration using Non-linear Least Squares

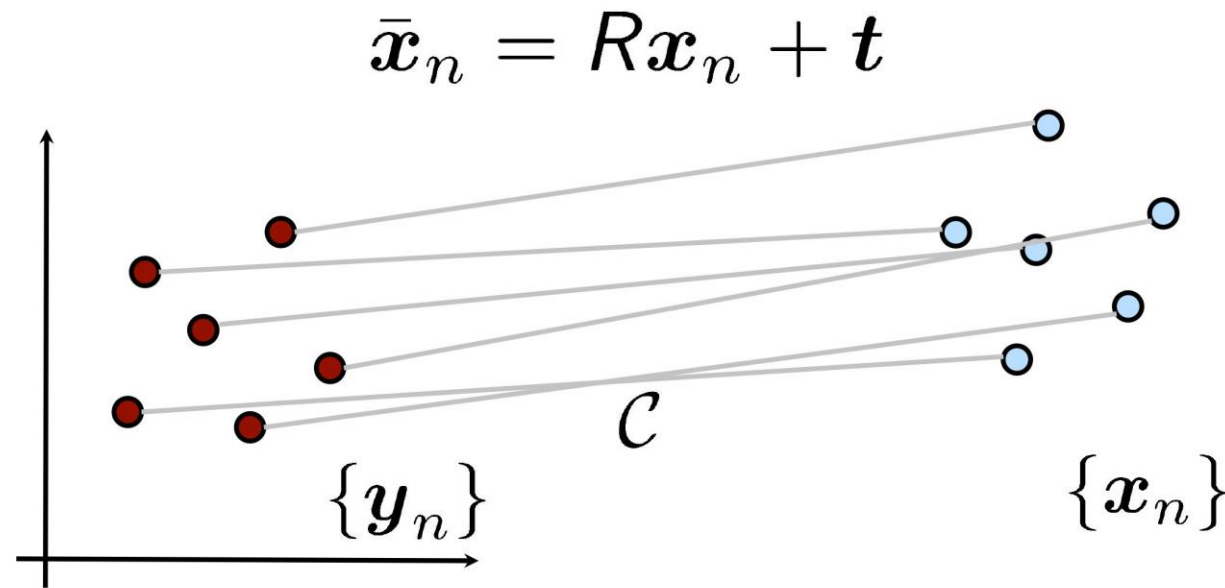
Simple Form of Point Cloud Registration



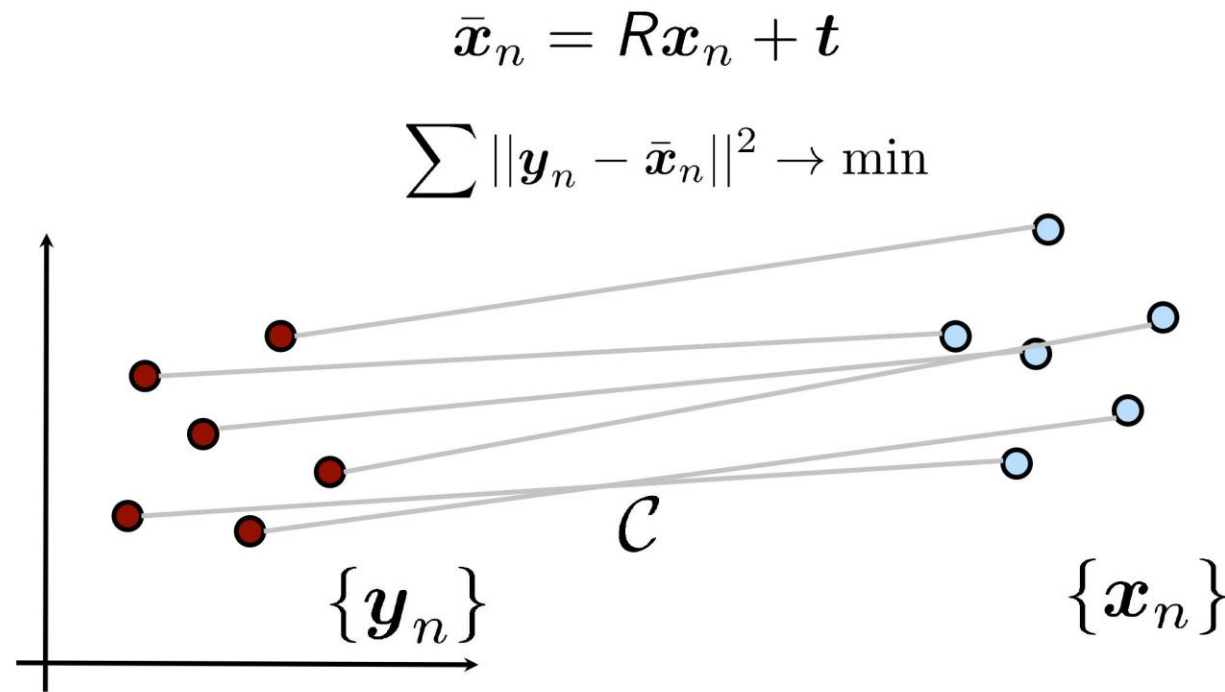
Simple Form of Point Cloud Registration



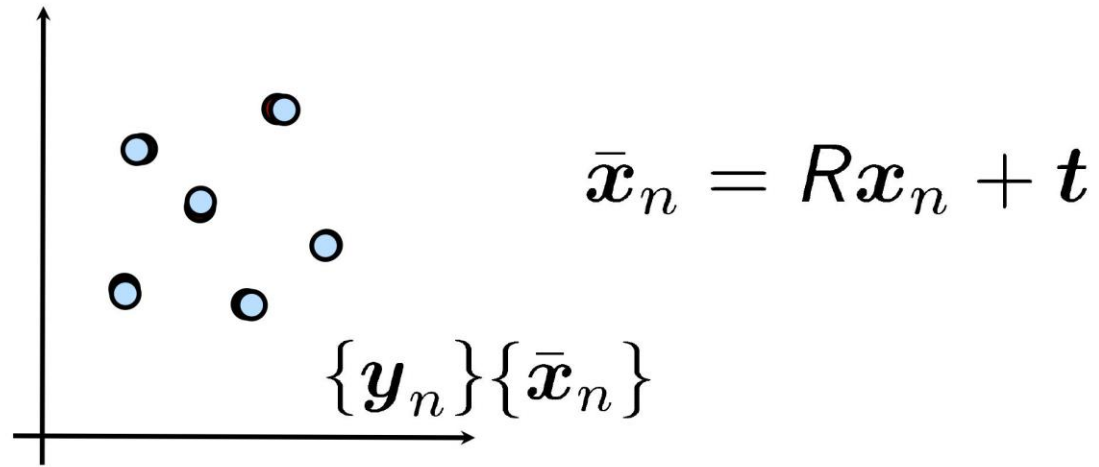
Simple Form of Point Cloud Registration



Simple Form of Point Cloud Registration



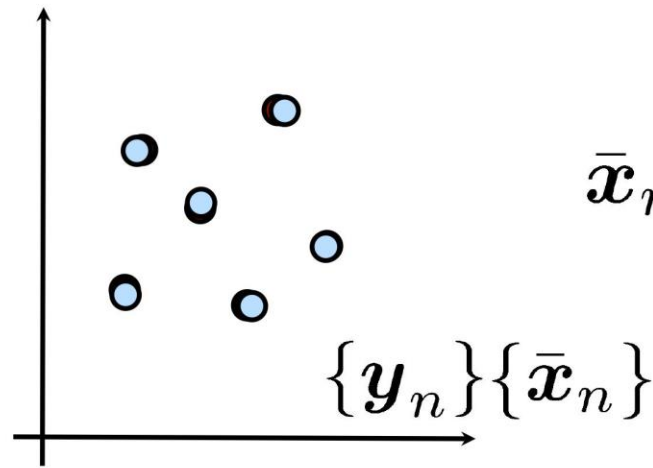
Simple Form of Point Cloud Registration



Simple Form of Point Cloud Registration

$$\sum ||\mathbf{y}_n - \bar{\mathbf{x}}_n||^2 \rightarrow \min$$

least squares solution!



$$\bar{\mathbf{x}}_n = R\mathbf{x}_n + t$$

Why a Least Squares Approach?

- SVD solution assumes point-to-point correspondences
- More complex error functions require a more general least squares approach
- LS approach can better consider uncertainties (3D point covariances)
- Often solved via an iterative Gauss Newton-based minimization

Start with Least Squares for 2D Point-to-Point Registration

Gauss Newton Minimization

- Example in 2D for point-to-point
- Objective: $\Phi(t_x, t_y, \theta) = \sum \|\bar{\mathbf{x}}_n - \mathbf{y}_n\|^2 \rightarrow \min$
- Error vector: $\mathbf{e}_n = \bar{\mathbf{x}}_n - \mathbf{y}_n$
- Expands to: $\mathbf{e}_n = R\mathbf{x}_n + \mathbf{t} - \mathbf{y}_n$
- Parameters: $[t_x, t_y, \theta]^\top$
- Explicitly: $\mathbf{e}_n(t_x, t_y, \theta) = R(\theta)\mathbf{x}_n + [t_x, t_y]^\top - \mathbf{y}_n$
- Linearize the non-linear error function

How does the Jacobian looks like?

Jacobian for 2D Points

- Computing the Jacobian

$$\begin{aligned} J_n(\mathbf{x}) &= \frac{\partial \mathbf{e}_n}{\partial \mathbf{x}} \\ &= \left[\frac{\partial \mathbf{e}_n}{\partial t_x}, \frac{\partial \mathbf{e}_n}{\partial t_y}, \frac{\partial \mathbf{e}_n}{\partial \theta} \right] \\ &= \begin{bmatrix} \frac{\partial \mathbf{e}_n^x}{\partial t_x}, \frac{\partial \mathbf{e}_n^x}{\partial t_y}, \frac{\partial \mathbf{e}_n^x}{\partial \theta} \\ \frac{\partial \mathbf{e}_n^y}{\partial t_x}, \frac{\partial \mathbf{e}_n^y}{\partial t_y}, \frac{\partial \mathbf{e}_n^y}{\partial \theta} \end{bmatrix} \end{aligned}$$

- leads to a 2 x 3 matrix in our case

Jacobian for 2D Points

- Computing the Jacobian for the error vector $e_n(t_x, t_y, \theta) = R(\theta) \mathbf{x}_n + [t_x, t_y]^\top - \mathbf{y}_n$

$$\begin{aligned} J_n &= \left[\frac{\partial e_n}{\partial t_x}, \frac{\partial e_n}{\partial t_y}, \frac{\partial e_n}{\partial \theta} \right] = \left[I; R'_\theta \mathbf{x}_n \right] \\ &= \begin{bmatrix} 1 & 0 & -\sin \theta & x_n - \cos \theta & y_n \\ 0 & 1 & \cos \theta & x_n - \sin \theta & y_n \end{bmatrix} \end{aligned}$$

- as

$$R'_\theta = \frac{\partial}{\partial \theta} \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} -\sin \theta & -\cos \theta \\ \cos \theta & -\sin \theta \end{bmatrix}$$

Gauss Newton Minimization

- Example in 2D for point-to-point
- Objective: $\Phi = \sum \|\bar{\mathbf{x}}_n - \mathbf{y}_n\|^2 \rightarrow \min$
- Error vector: $\mathbf{e}_n = R\mathbf{x}_n + \mathbf{t} - \mathbf{y}_n$
- Jacobian: $J_n = \begin{bmatrix} 1 & 0 & -\sin \theta & x_n - \cos \theta & y_n \\ 0 & 1 & \cos \theta & x_n - \sin \theta & y_n \end{bmatrix}$

Todo

- Compute normal equation matrix
- Compute right-hand side
- Solve resulting linear system

Gauss Newton Minimization

- Jacobian: $J_n = \begin{bmatrix} 1 & 0 & -\sin \theta & x_n - \cos \theta & y_n \\ 0 & 1 & \cos \theta & x_n - \sin \theta & y_n \end{bmatrix}$

- Matrix: $H_n = J_n^\top J_n$

- Right-hand side: $b_n = J_n^\top e_n$

- Compute terms over all points

$$H = \sum_n H_n = \sum_n J_n^\top J_n \quad b = \sum_n b_n = \sum_n J_n^\top e_n$$

- Solve $H\Delta x = -b$ via $\Delta x = -H^{-1}b$

- Update parameters $x \leftarrow x + \Delta x$

- Iterative until convergence

2D Least Squares Example

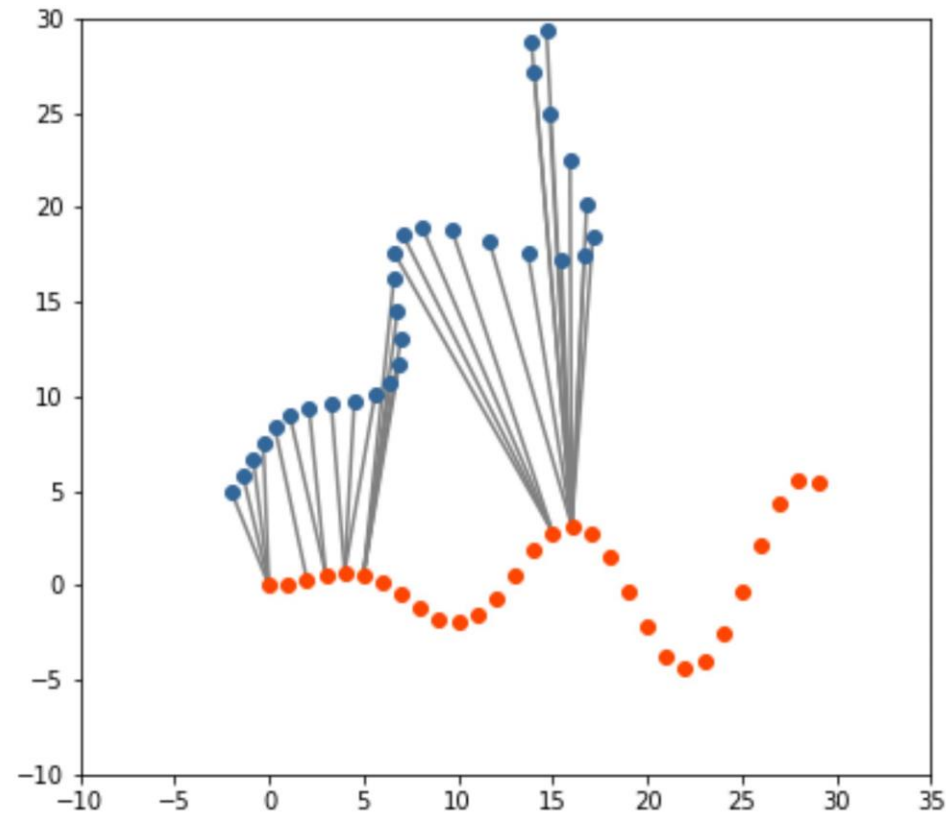


Image courtesy: Bogoslavskyi

2D Least Squares Example

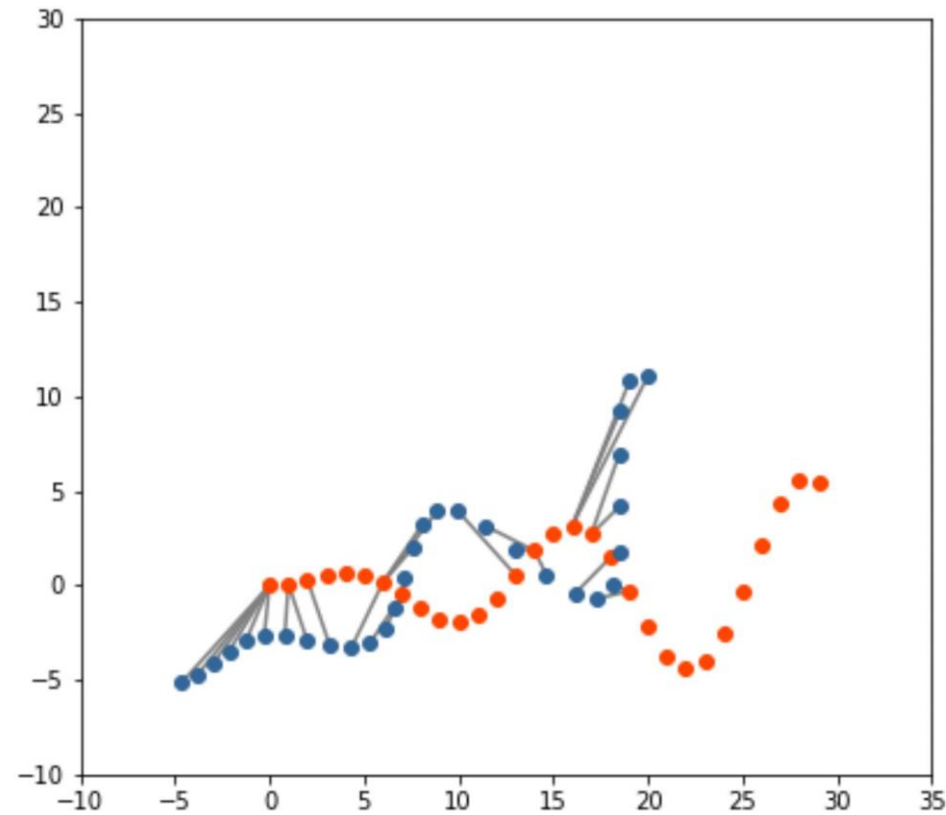


Image courtesy: Bogoslavskyi

2D Least Squares Example

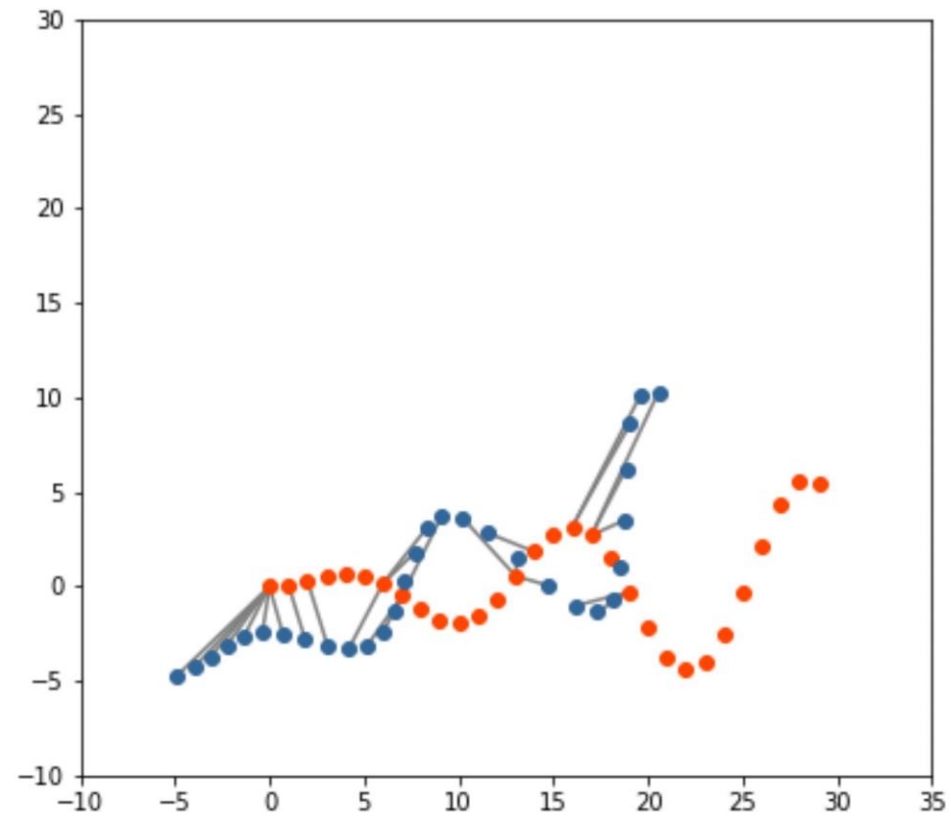


Image courtesy: Bogoslavskyi

2D Least Squares Example

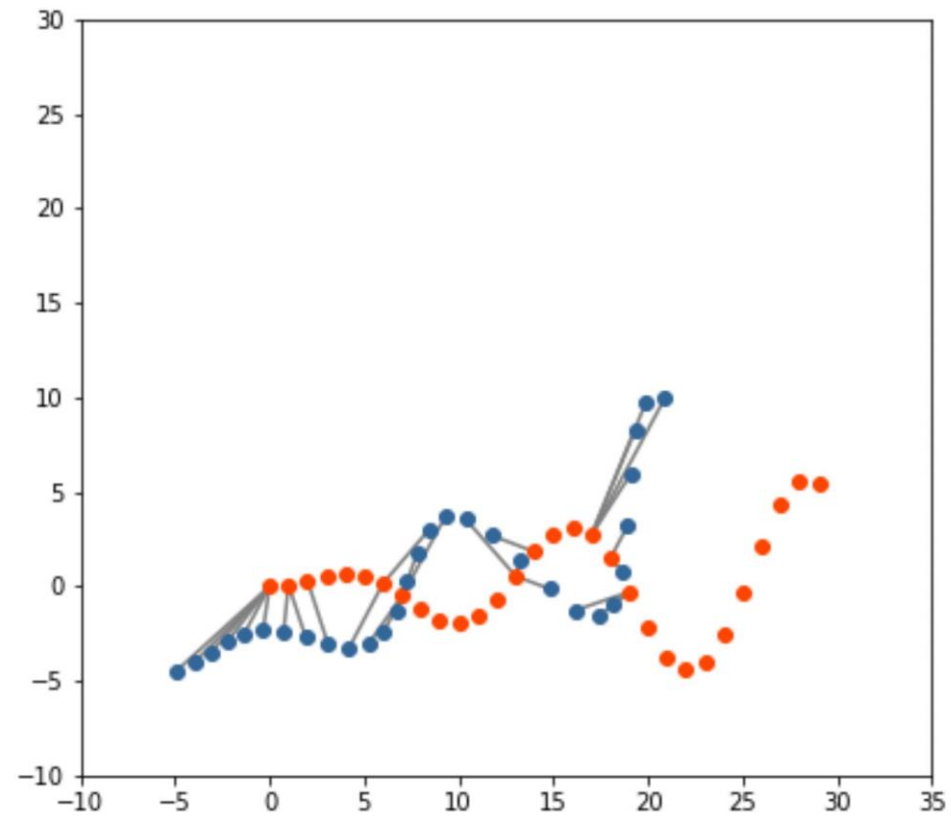


Image courtesy: Bogoslavskyi

2D Least Squares Example

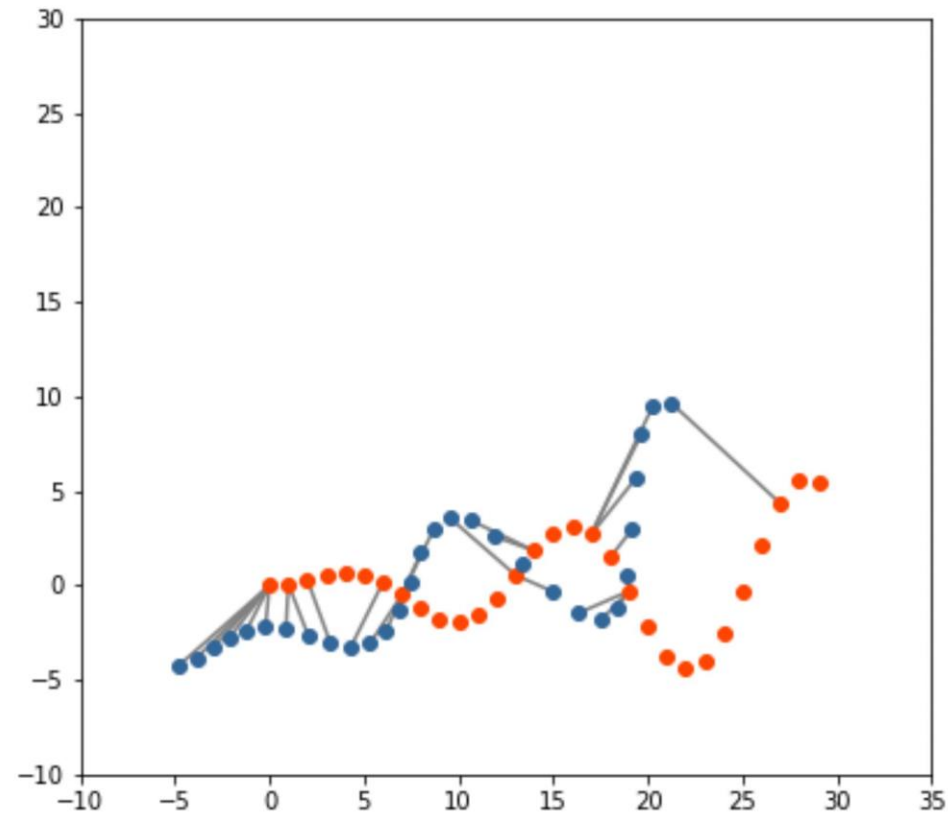


Image courtesy: Bogoslavskyi

2D Least Squares Example

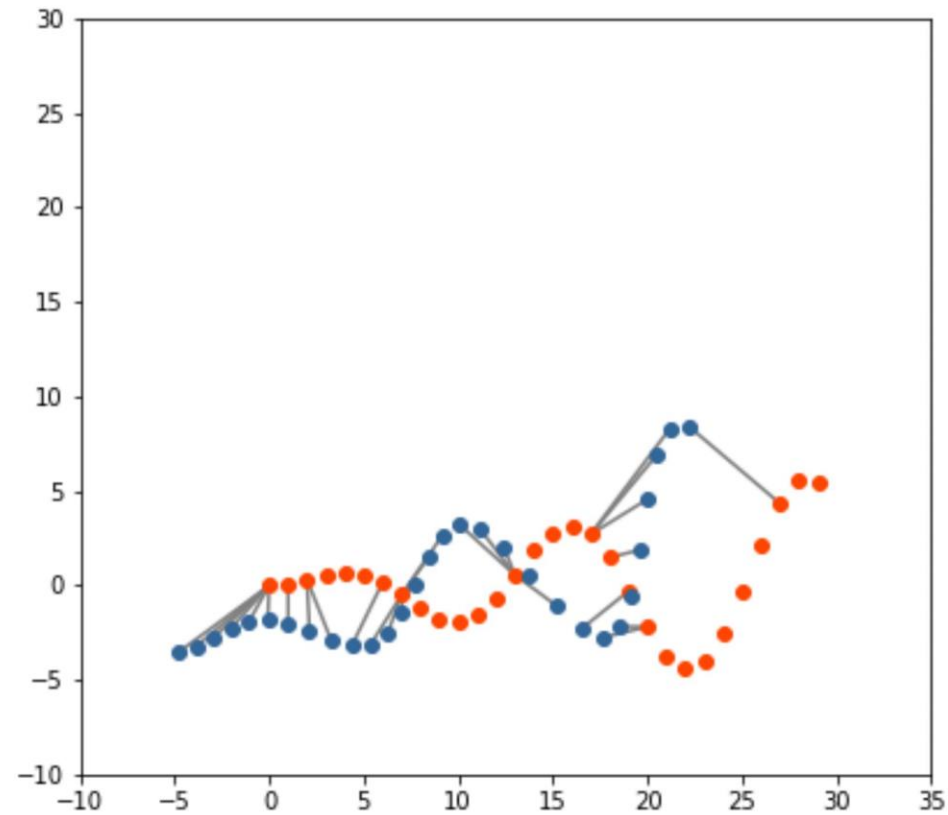


Image courtesy: Bogoslavskyi

2D Least Squares Example

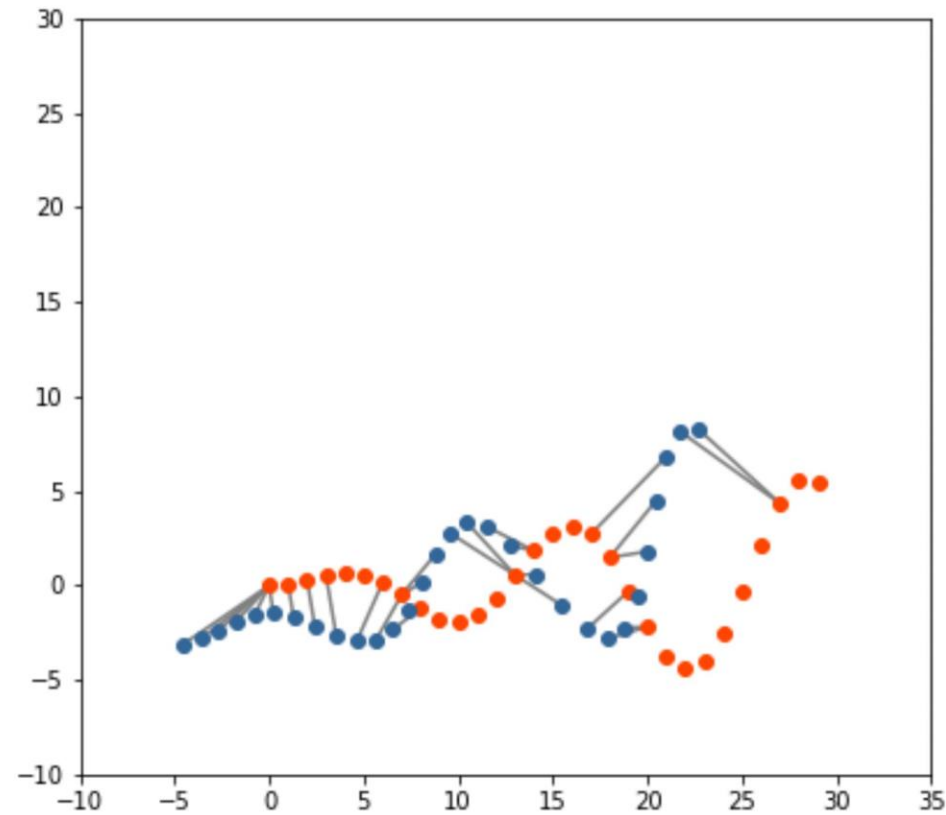


Image courtesy: Bogoslavskyi

2D Least Squares Example

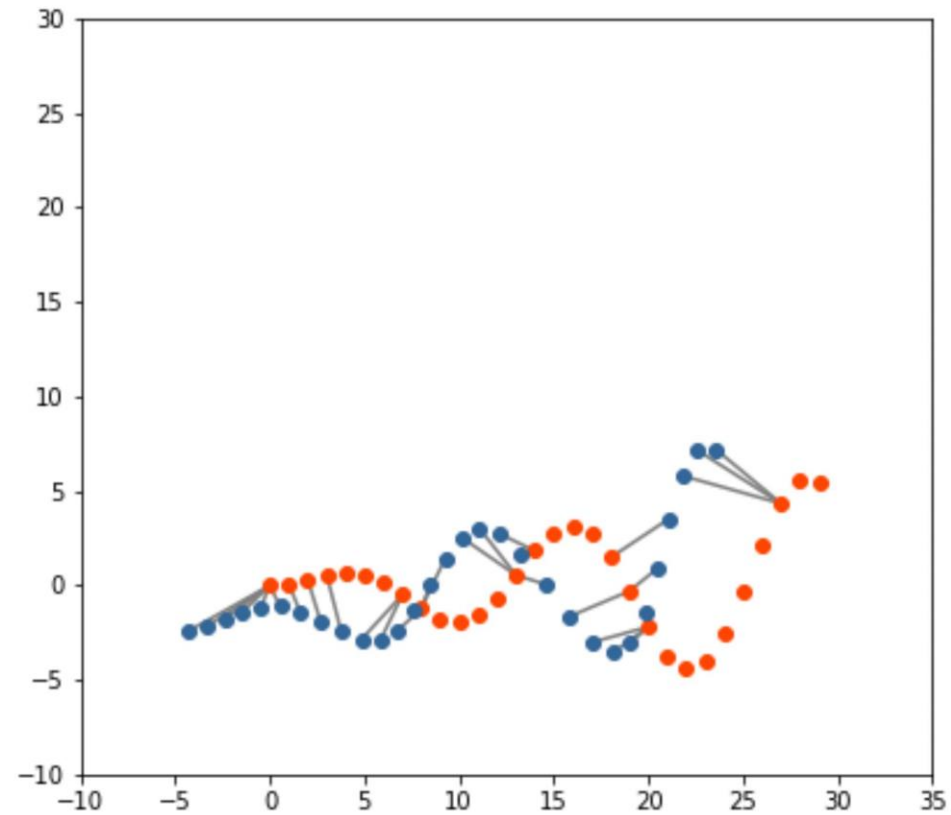


Image courtesy: Bogoslavskyi

2D Least Squares Example

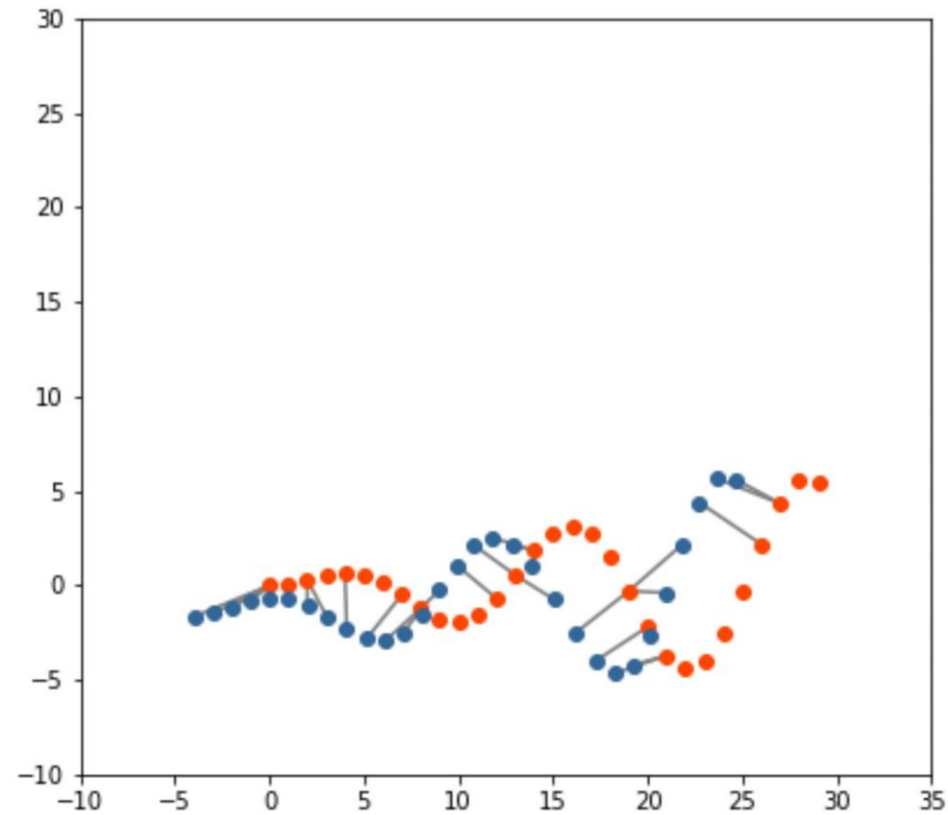


Image courtesy: Bogoslavskyi

2D Least Squares Example

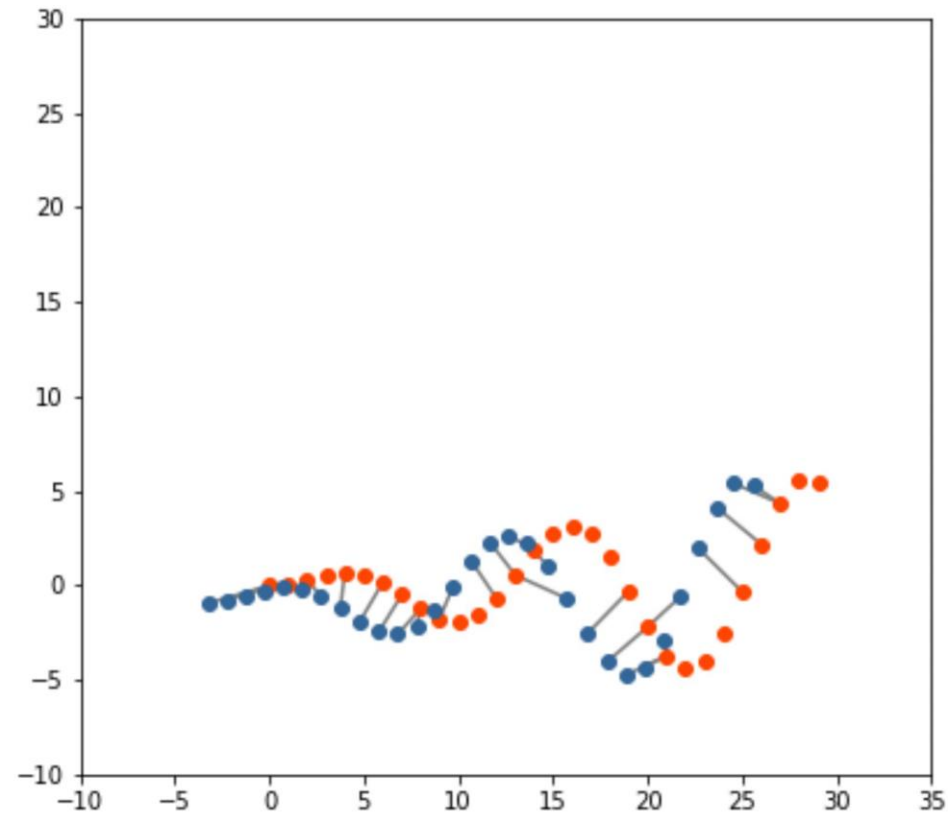


Image courtesy: Bogoslavskyi

2D Least Squares Example

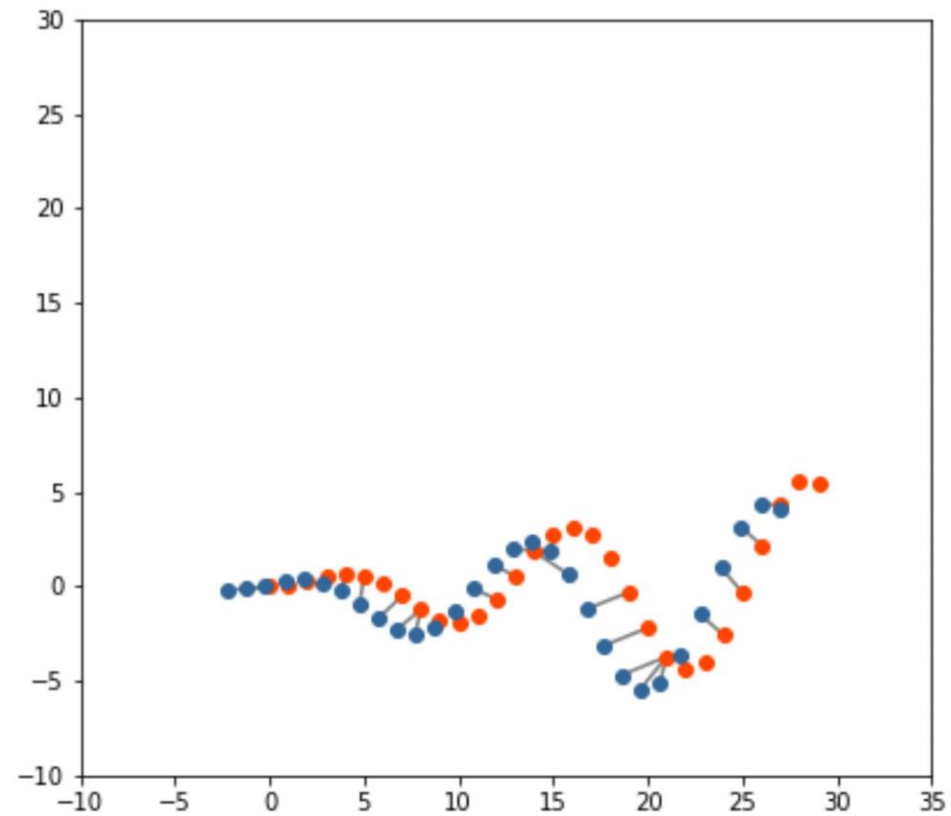


Image courtesy: Bogoslavskyi

2D Least Squares Example

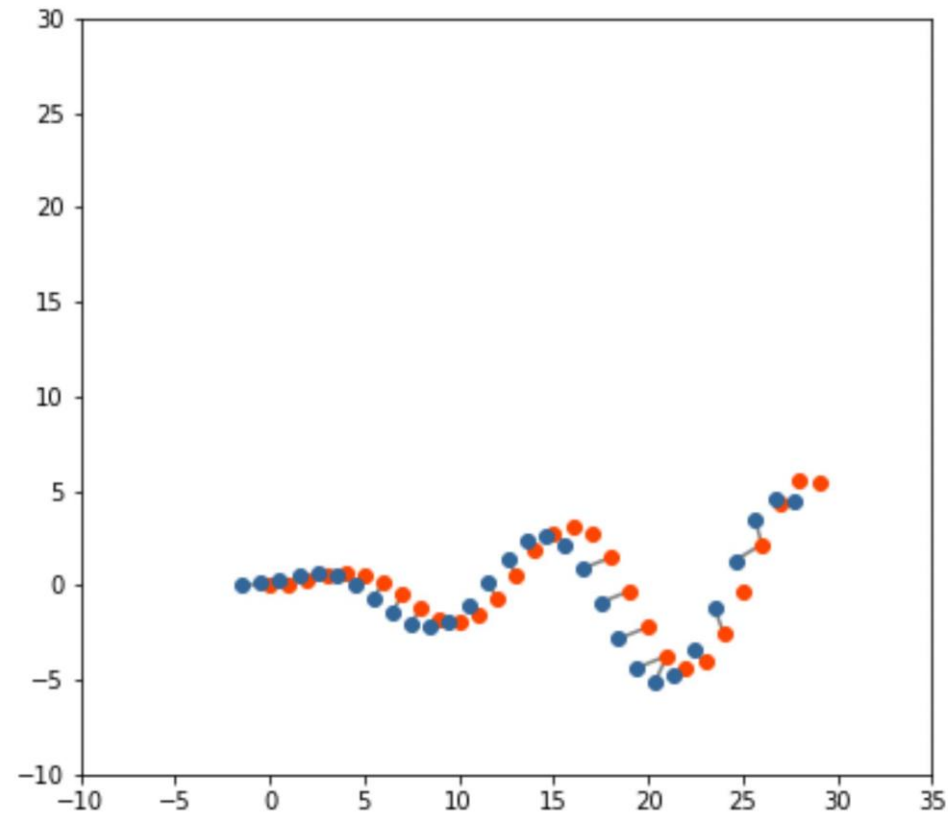


Image courtesy: Bogoslavskyi

2D Least Squares Example

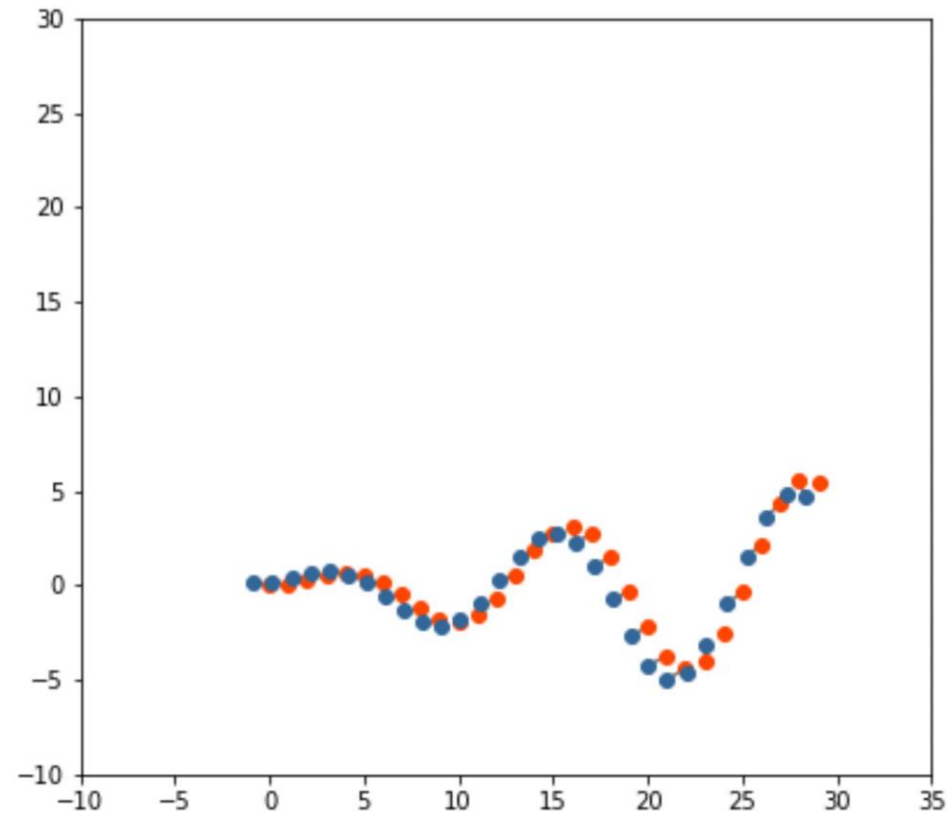


Image courtesy: Bogoslavskyi

2D Least Squares Example

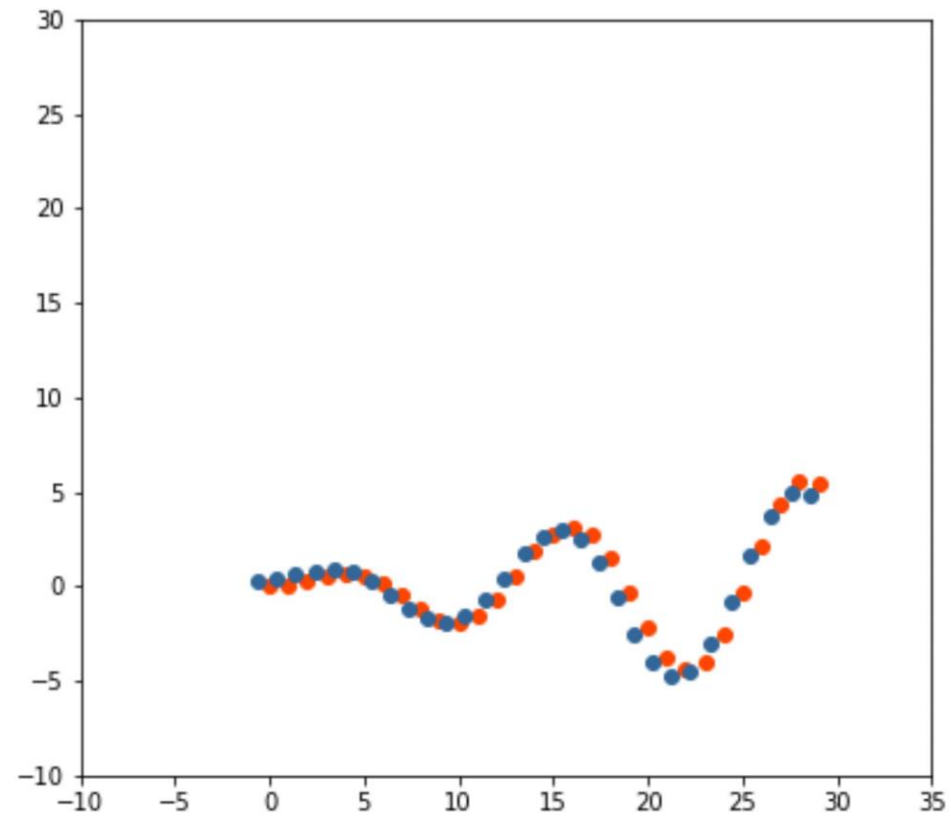


Image courtesy: Bogoslavskyi

2D Least Squares Example

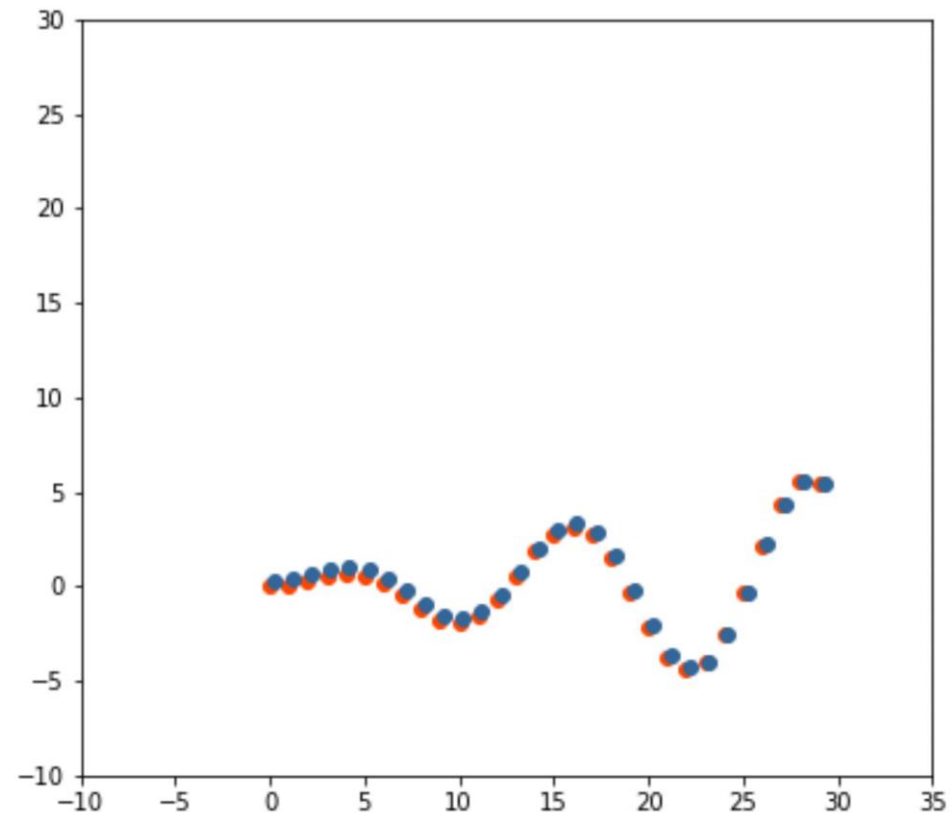


Image courtesy: Bogoslavskyi

2D Least Squares Example

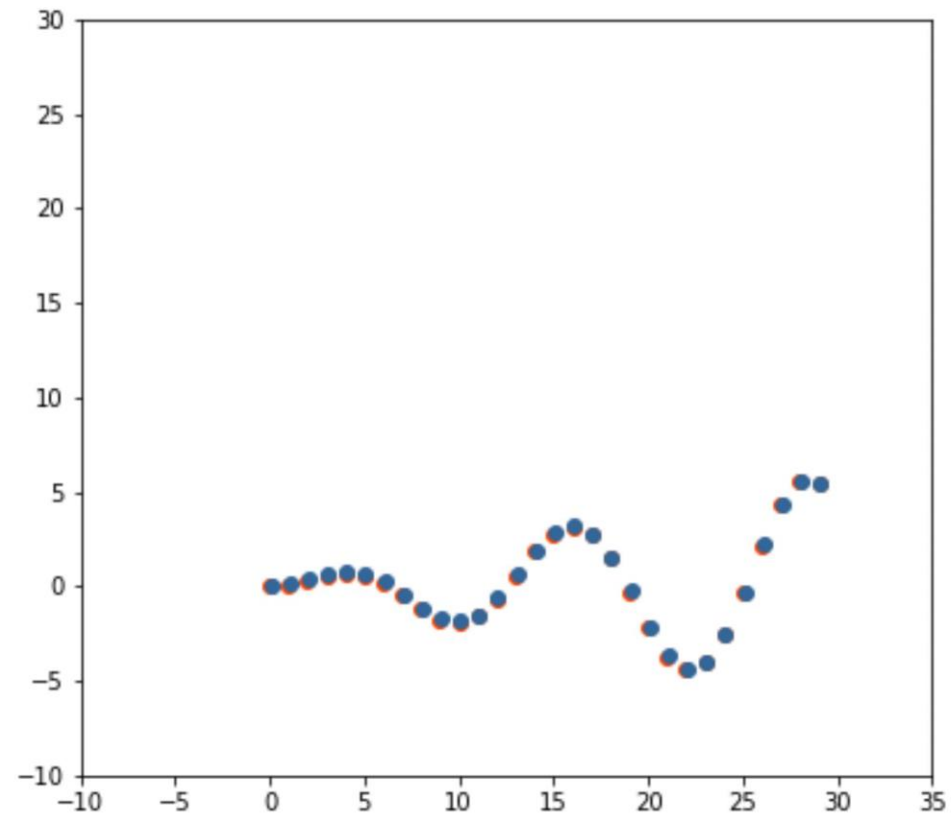


Image courtesy: Bogoslavskyi

2D Least Squares Example

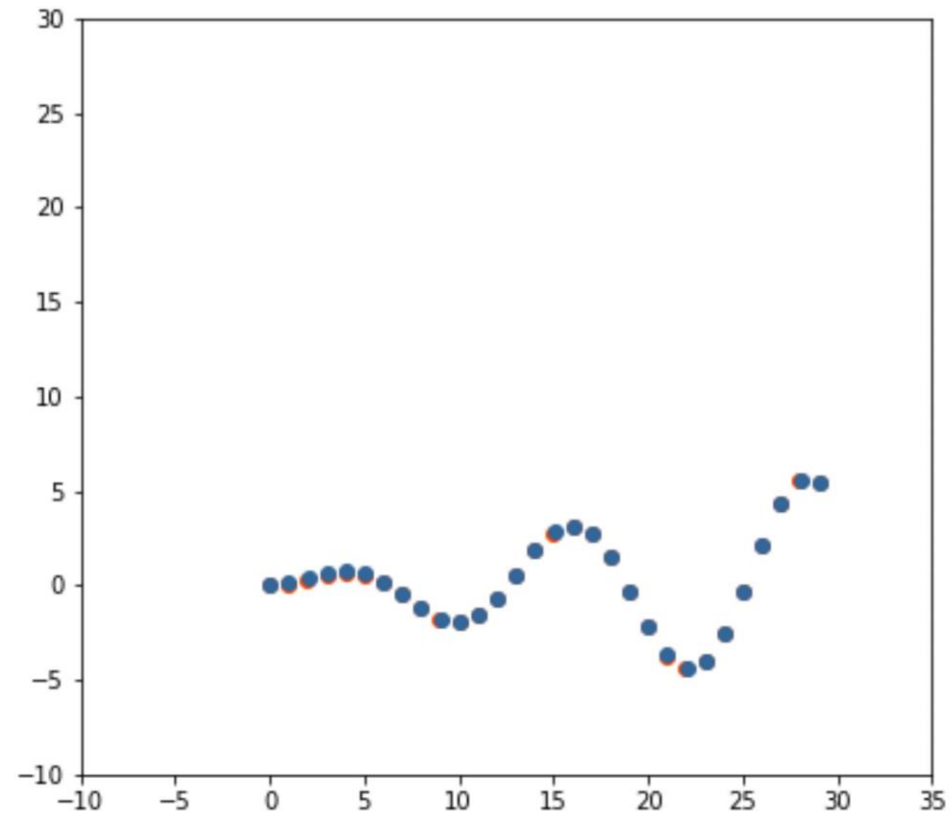


Image courtesy: Bogoslavskyi

2D Least Squares Example

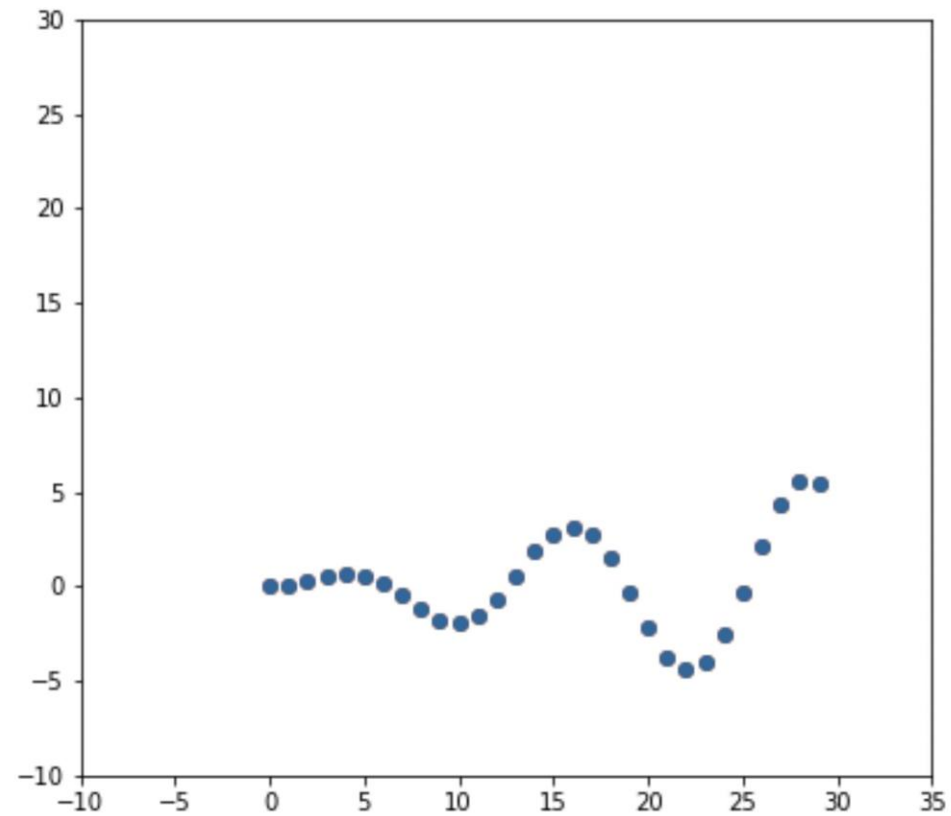
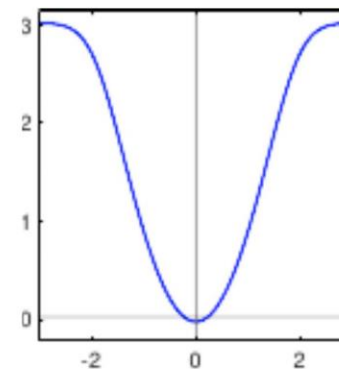
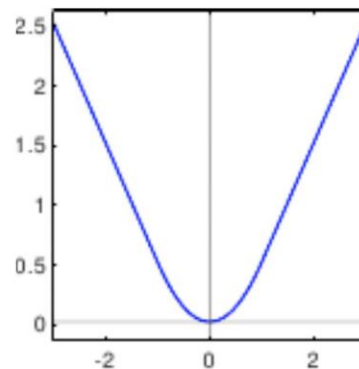
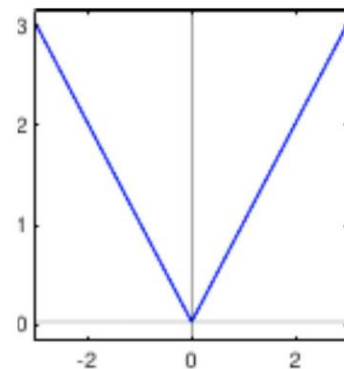


Image courtesy: Bogoslavskyi

Robust Least Squares

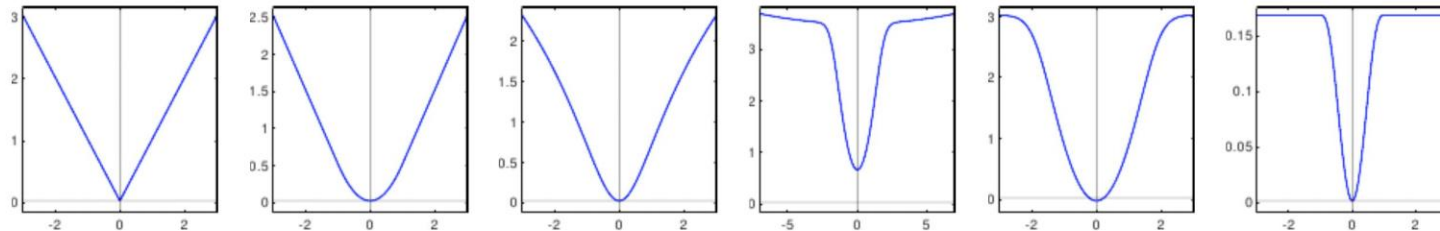
Robust Least Squares

- Data association outliers strongly impact the least squares result
- Robust kernels / M-estimators aims at down-weighting the impact of outliers
- Function that changes the error function depending on its magnitude

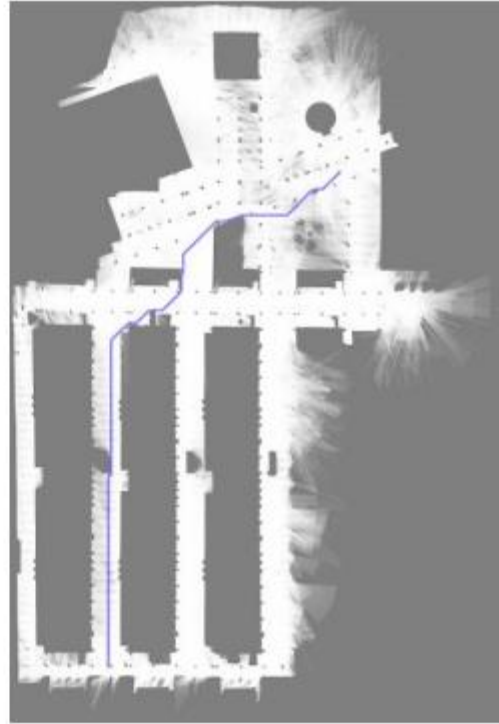
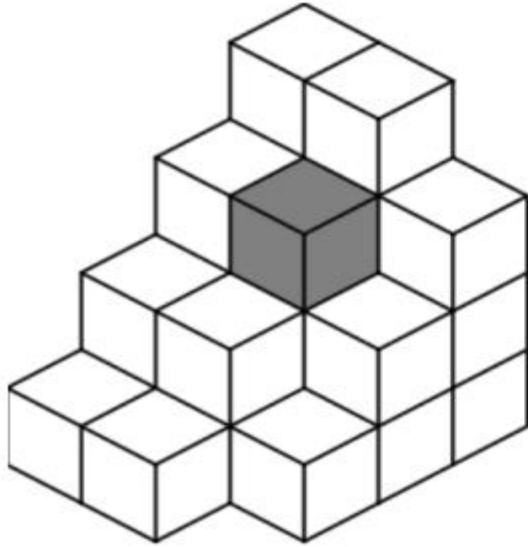


Robust Least Squares

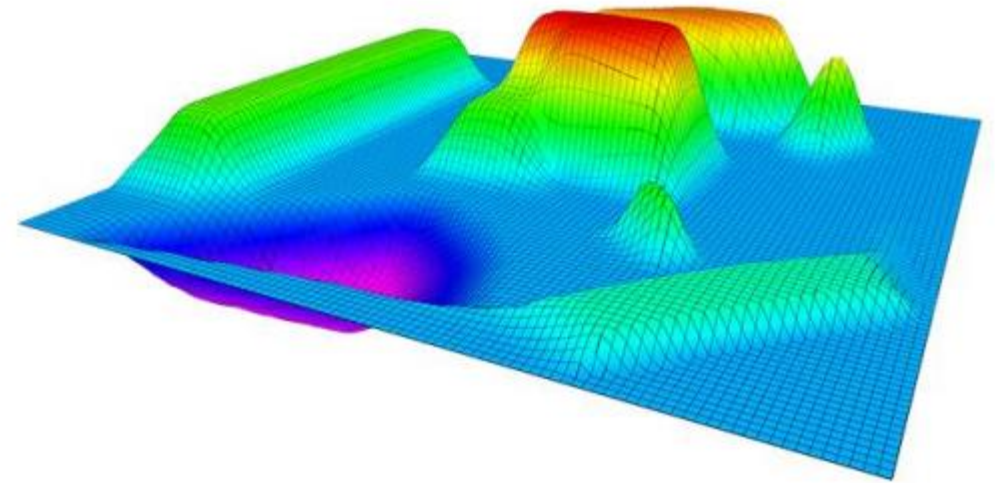
- Weighted least squares approach to realize robust least squares estimation
- Each kernel yields a specific weight
- The kernel will impact the Jacobians
- The rest stays the same
- The choice of the kernel must align with the outlier distribution



Map Representation for UAV navigation



- Most Dense
- Structural
- Direct Index Query



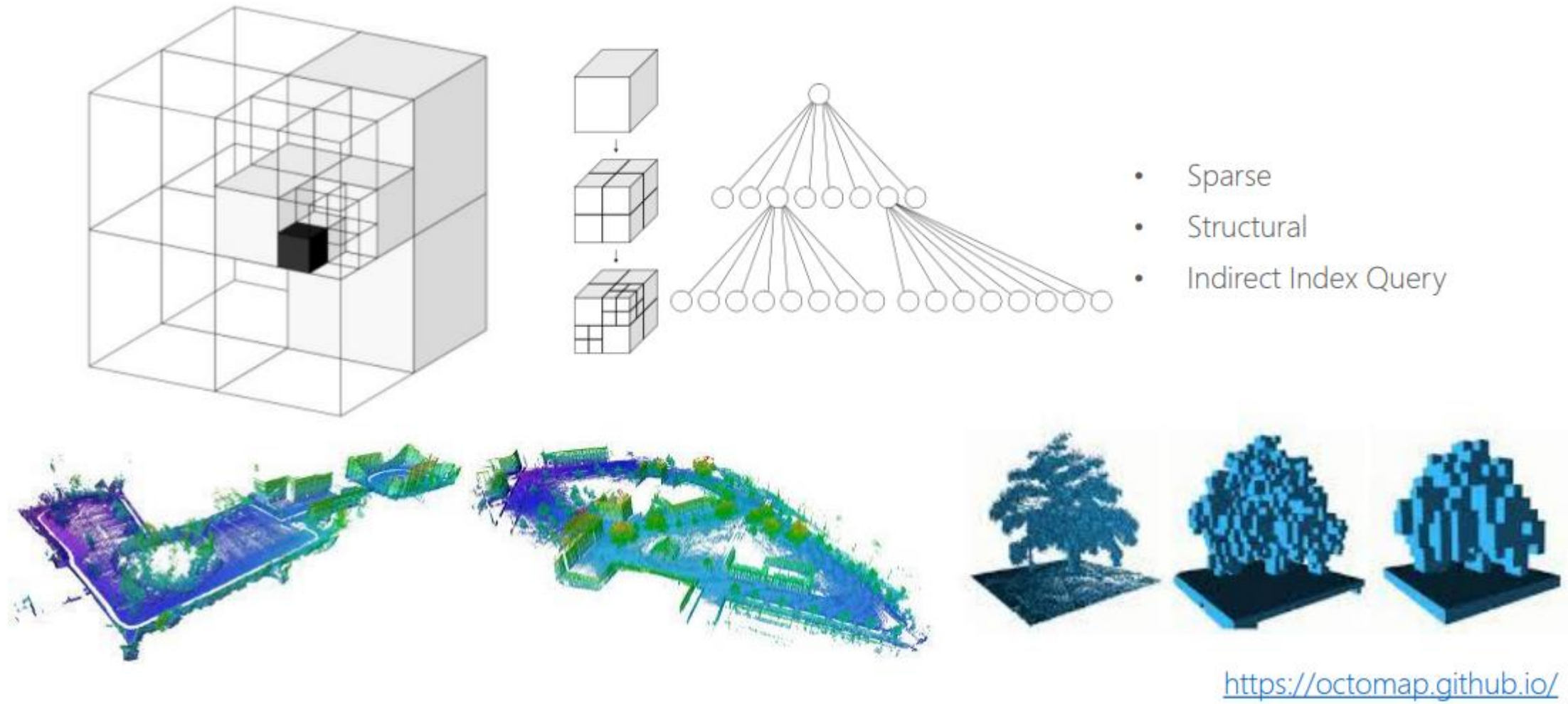
Occupancy Grid Map

Map Representation for UAV navigation



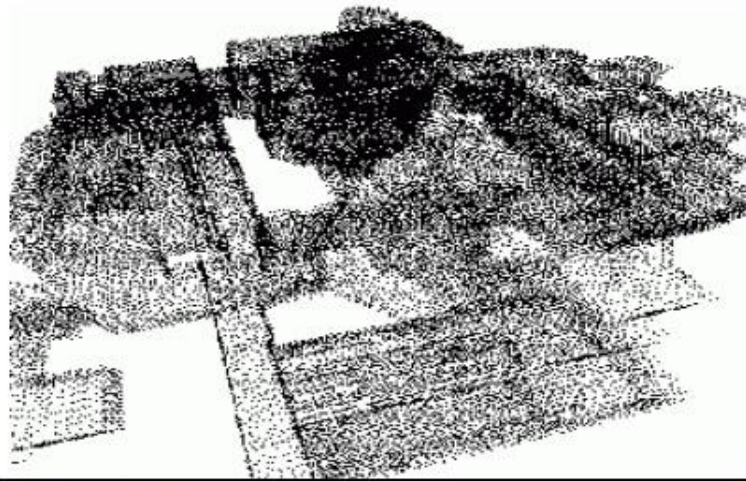
Occupancy Grid Map

Map Representation for UAV navigation

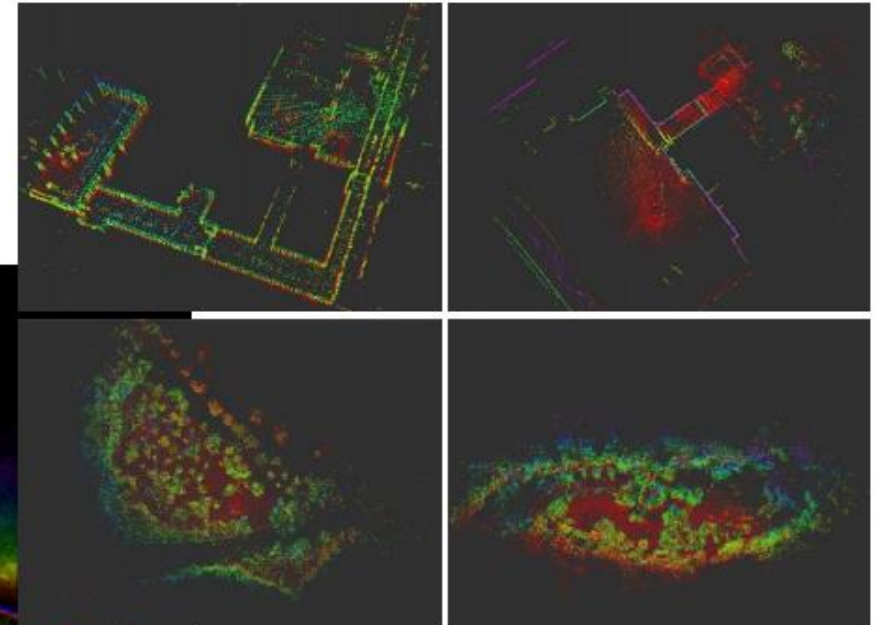
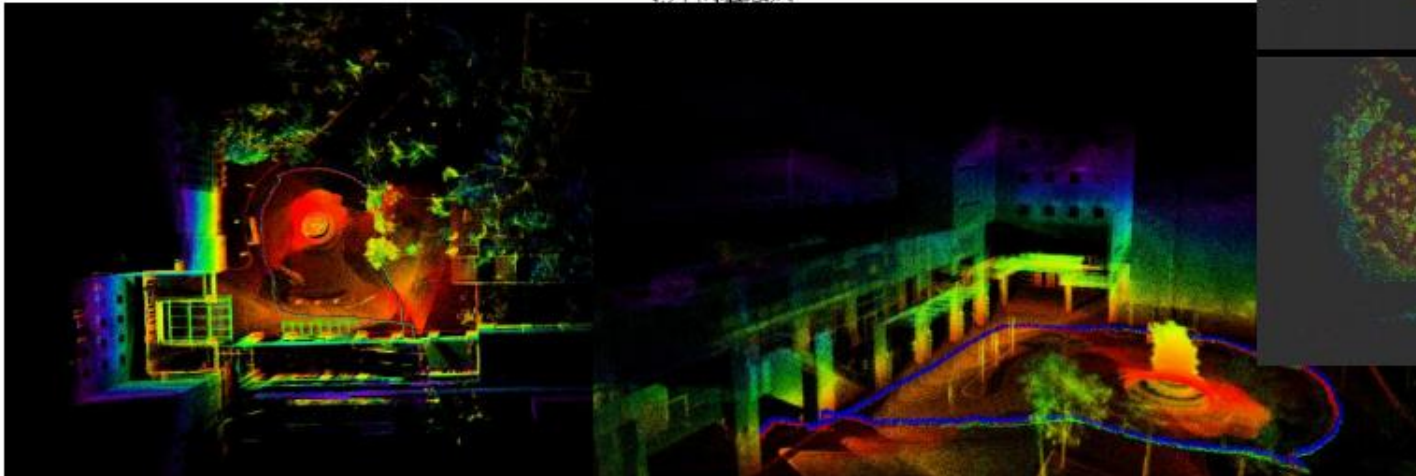


OctoMap

Map Representation for UAV navigation



- Un-ordered
- No Index Query



PCL
<http://pointclouds.org/>

Point Cloud

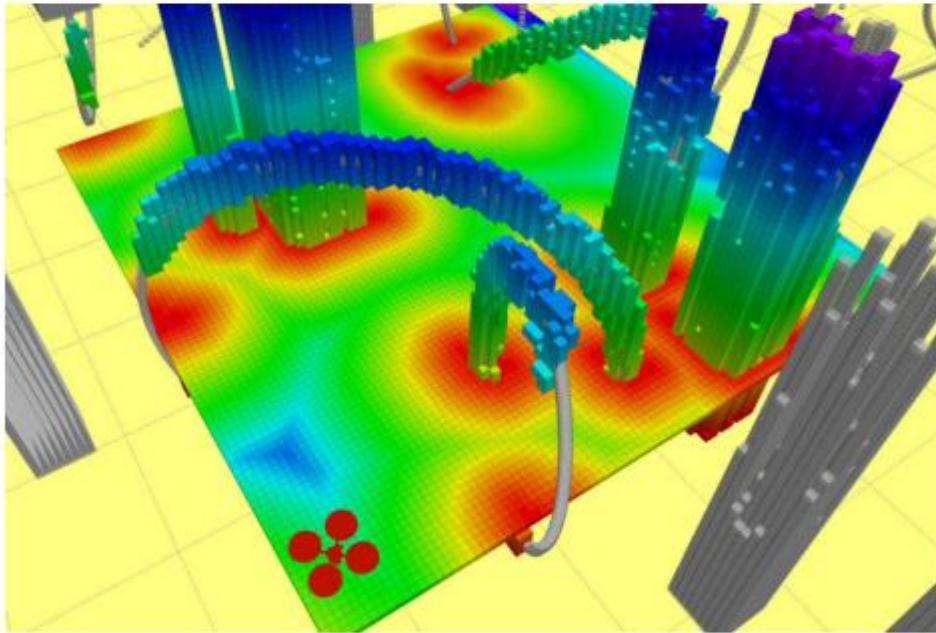
Map Representation for UAV navigation



Point Cloud

Map Representation for UAV navigation

Batch Update, Local Map



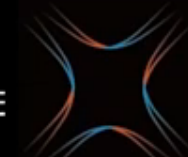
Distance Transforms of Sampled Functions, PF Felzenszwalb

RAPTOR: Robust and Perception-aware Trajectory Replanning for Quadrotor Fast Flight

Boyuan Zhou, Jie Pan, Fei Gao and Shaojie Shen



香港科技大學
THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

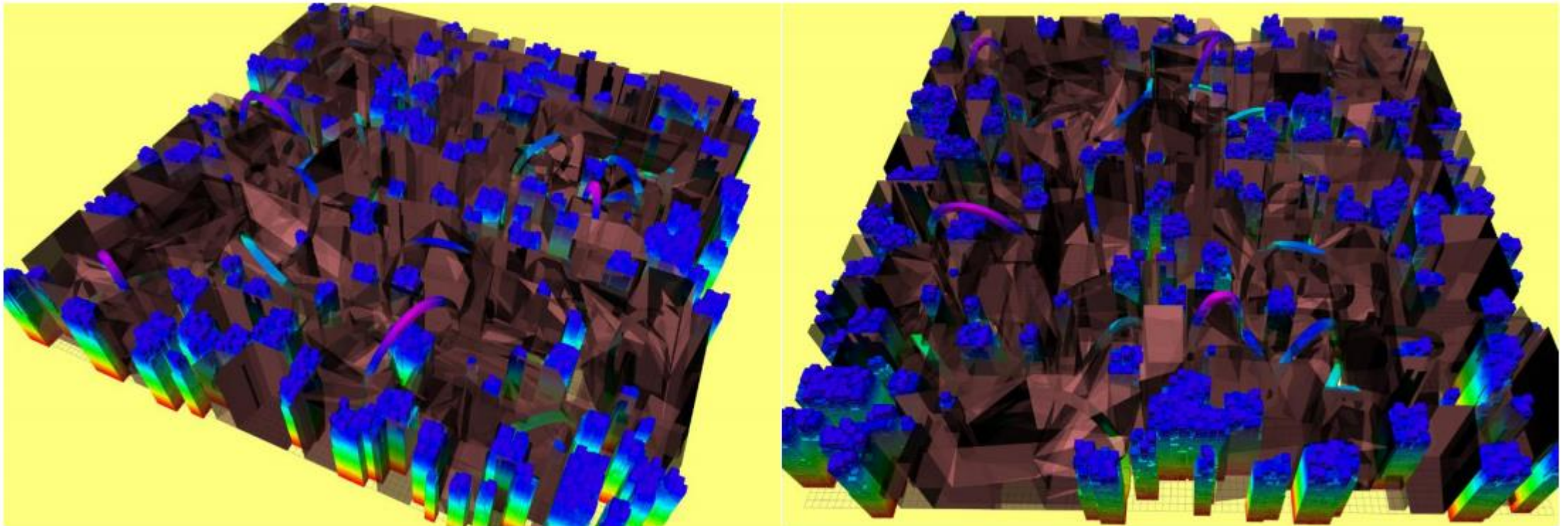


香港科技大學-
大疆創新科技聯合實驗室
HKUST-DJI JOINT
INNOVATION LABORATORY

Euclidean Signed Distance Field(ESDF)

Map Representation for UAV navigation

Free-space Roadmap



<https://github.com/HKUST-Aerial-Robotics/Teach-Repeat-Replan>

Map Representation for UAV navigation

Geometrically Constrained Trajectory Optimization
for Multicopters

Video: Real-World SE(3) Planning Powered by Our Framework

Zhepei Wang, Xin Zhou, Chao Xu and Fei Gao
ZJU FAST Lab



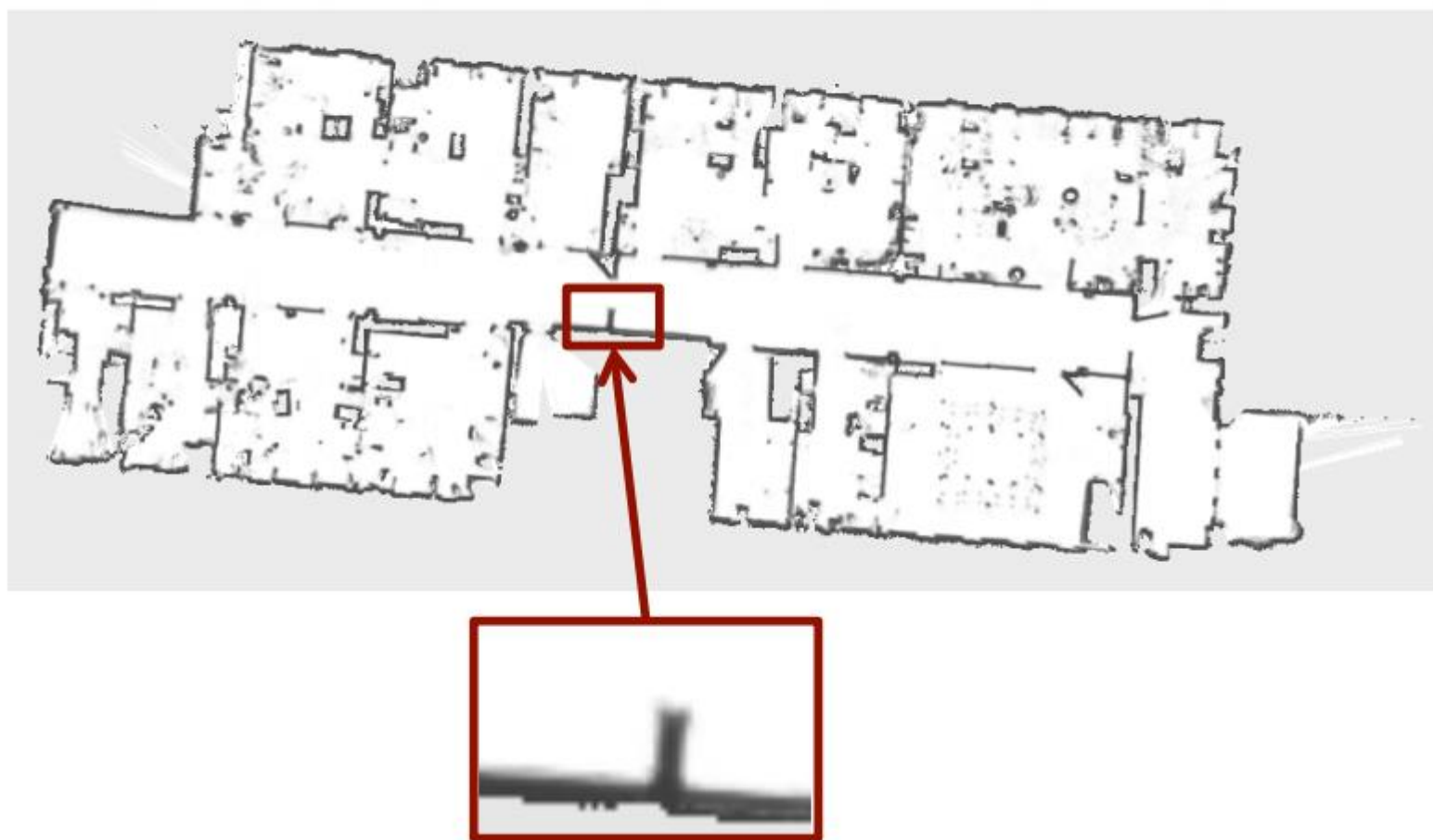
Institute of Cyber-Systems and Control
Zhejiang University

Occupancy Grid Map

Grid Maps

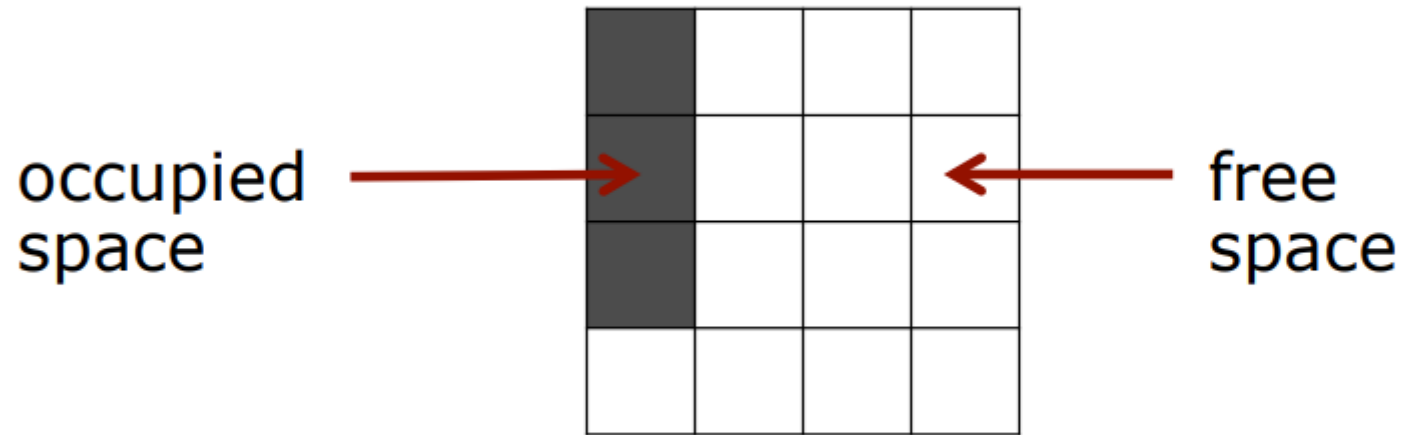
- Discretize the world into cells
- Grid structure is rigid
- Each cell is assumed to be occupied or free space
- Non-parametric model
- Large maps require substantial memory resources
- Do not rely on a feature detector

Example



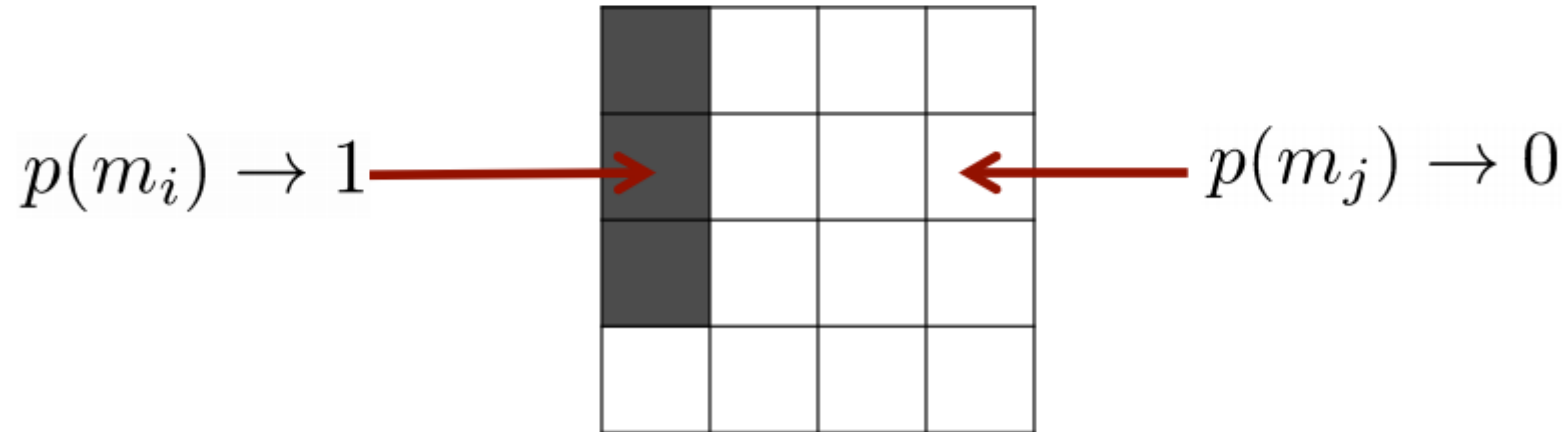
Assumption 1

- The area that corresponds to a cell is either completely free or occupied



Representation

- Each cell is a **binary random variable** that models the occupancy

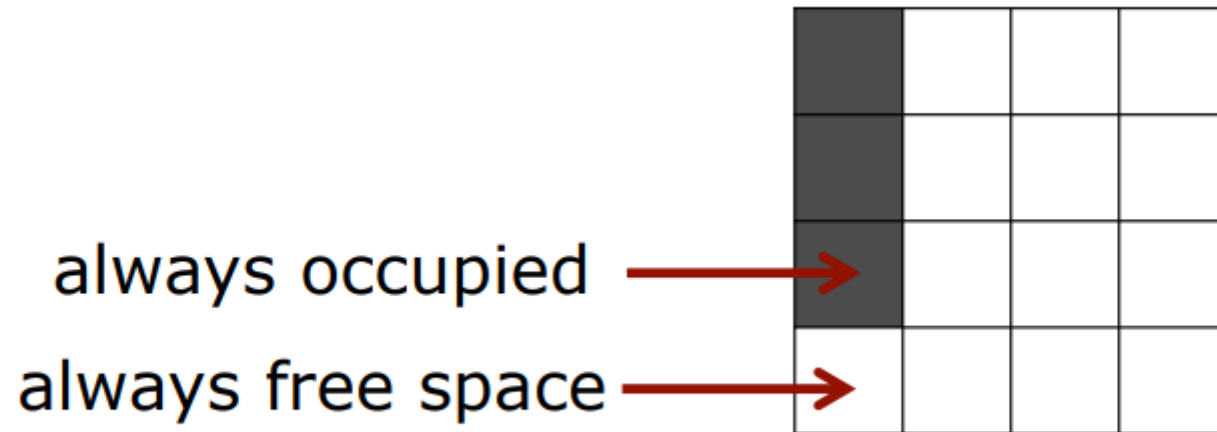


Occupancy Probability

- Each cell is a **binary random variable** that models the occupancy
- Cell is occupied: $p(m_i) = 1$
- Cell is not occupied: $p(m_i) = 0$
- No knowledge: $p(m_i) = 0.5$

Assumption 2

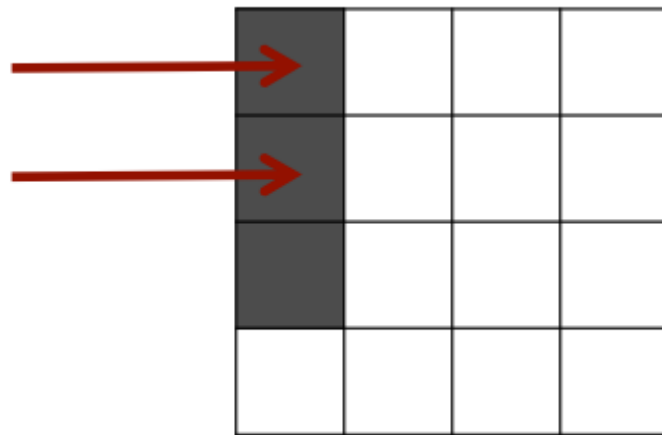
- The world is **static** (most mapping systems make this assumption)



Assumption 3

- The cells (the random variables) are **independent** of each other

no dependency
between the cells



Representation

- The probability distribution of the map is given by the product over the cells

$$p(m) = \prod_i p(m_i)$$

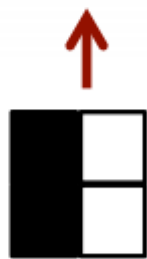

map

cell

Representation

- The probability distribution of the map is given by the product over the cells

$$p(m) = \prod_i p(m_i)$$



example map
(4-dim state)



4 individual cells

Representation

m_0	m_1
m_2	m_3

$$\begin{aligned}p(m_0) &= 0.9 \\p(m_1) &= 0.5 \\p(m_2) &= 0.8 \\p(m_3) &= 0.3\end{aligned}$$

m_0	m_1
m_2	m_3

$$\begin{aligned}p(m) &= \\&0.9 \times (1 - 0.5) \times \\&0.8 \times (1 - 0.3) \\&= 0.252\end{aligned}$$

m_0	m_1
m_2	m_3

$$\begin{aligned}p(m) &= \\&(1 - 0.9) \times 0.5 \times \\&(1 - 0.8) \times 0.3 \\&= 0.003\end{aligned}$$

Estimating a Map From Data

- Given sensor data $z_{1:t}$ and the poses $x_{1:t}$ of the sensor, estimate the map

$$p(m \mid z_{1:t}, x_{1:t}) = \prod_i p(m_i \mid z_{1:t}, x_{1:t})$$

binary random variable

➡ Binary Bayes filter
(for a static state)

Static State Binary Bayes Filter

$$p(m_i \mid z_{1:t}, x_{1:t}) \stackrel{\text{Bayes rule}}{=} \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})}$$

Bayes rule: $P(A|B) = P(B|A)P(A)/P(B)$

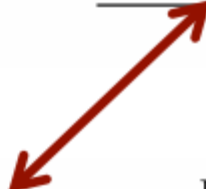
Static State Binary Bayes Filter

$$\begin{array}{lcl}
 p(m_i \mid z_{1:t}, x_{1:t}) & \stackrel{\text{Bayes rule}}{=} & \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \stackrel{\text{Markov}}{=} & \frac{p(z_t \mid m_i, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(z_t \mid z_{1:t-1}, x_{1:t})}
 \end{array}$$

The diagram illustrates the simplification of the Bayes filter equation using the Markov property. Red arrows point from the terms in the Markov equation to their corresponding terms in the Bayes rule equation:

- A red arrow points from $p(z_t \mid m_i, x_t)$ in the Markov equation to $p(z_t \mid m_i, z_{1:t-1}, x_{1:t})$ in the Bayes rule equation.
- A red arrow points from $p(m_i \mid z_{1:t-1}, x_{1:t-1})$ in the Markov equation to $p(m_i \mid z_{1:t-1}, x_{1:t})$ in the Bayes rule equation.

Static State Binary Bayes Filter

$$\begin{aligned}
 p(m_i \mid z_{1:t}, x_{1:t}) &\stackrel{\text{Bayes rule}}{=} \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 &\stackrel{\text{Markov}}{=} \frac{p(z_t \mid m_i, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 p(z_t \mid m_i, x_t) &\stackrel{\text{Bayes rule}}{=} \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t)}{p(m_i \mid x_t)}
 \end{aligned}$$


Static State Binary Bayes Filter

$$\begin{aligned} p(m_i \mid z_{1:t}, x_{1:t}) & \stackrel{\text{Bayes rule}}{=} \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\ & \stackrel{\text{Markov}}{=} \frac{p(z_t \mid m_i, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\ & \stackrel{\text{Bayes rule}}{=} \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i \mid x_t) p(z_t \mid z_{1:t-1}, x_{1:t})} \end{aligned}$$

Static State Binary Bayes Filter

$$\begin{array}{ll}
 p(m_i \mid z_{1:t}, x_{1:t}) & \begin{array}{l} \text{Bayes rule} \\ \underline{\underline{=}} \end{array} \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \begin{array}{l} \text{Markov} \\ \underline{\underline{=}} \end{array} \frac{p(z_t \mid m_i, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \begin{array}{l} \text{Bayes rule} \\ \underline{\underline{=}} \end{array} \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i \mid x_t) p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \begin{array}{l} \text{Markov} \\ \underline{\underline{=}} \end{array} \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i) p(z_t \mid z_{1:t-1}, x_{1:t})}
 \end{array}$$


Static State Binary Bayes Filter

$$\begin{array}{ll}
 p(m_i \mid z_{1:t}, x_{1:t}) & \text{Bayes rule} \quad \frac{p(z_t \mid m_i, z_{1:t-1}, x_{1:t}) p(m_i \mid z_{1:t-1}, x_{1:t})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \text{Markov} \quad \frac{p(z_t \mid m_i, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \text{Bayes rule} \quad \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i \mid x_t) p(z_t \mid z_{1:t-1}, x_{1:t})} \\
 & \text{Markov} \quad \frac{p(m_i \mid z_t, x_t) p(z_t \mid x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i) p(z_t \mid z_{1:t-1}, x_{1:t})}
 \end{array}$$

Do exactly the same for the opposite event:

$$p(\neg m_i \mid z_{1:t}, x_{1:t}) \quad \text{the same} \quad \frac{p(\neg m_i \mid z_t, x_t) p(z_t \mid x_t) p(\neg m_i \mid z_{1:t-1}, x_{1:t-1})}{p(\neg m_i) p(z_t \mid z_{1:t-1}, x_{1:t})}$$

Static State Binary Bayes Filter

- By computing the ratio of both probabilities, we obtain:

$$\frac{p(m_i \mid z_{1:t}, x_{1:t})}{p(\neg m_i \mid z_{1:t}, x_{1:t})} = \frac{\frac{p(m_i \mid z_t, x_t) \cancel{p(z_t \mid x_t)} p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i) \cancel{p(z_t \mid z_{1:t-1}, x_{1:t})}}}{\frac{p(\neg m_i \mid z_t, x_t) \cancel{p(z_t \mid x_t)} p(\neg m_i \mid z_{1:t-1}, x_{1:t-1})}{p(\neg m_i) \cancel{p(z_t \mid z_{1:t-1}, x_{1:t})}}}$$

Static State Binary Bayes Filter

- By computing the ratio of both probabilities, we obtain:

$$\begin{aligned} & \frac{p(m_i \mid z_{1:t}, x_{1:t})}{p(\neg m_i \mid z_{1:t}, x_{1:t})} \\ &= \frac{p(m_i \mid z_t, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1}) p(\neg m_i)}{p(\neg m_i \mid z_t, x_t) p(\neg m_i \mid z_{1:t-1}, x_{1:t-1}) p(m_i)} \\ &= \frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)} \frac{p(m_i \mid z_{1:t-1}, x_{1:t-1})}{1 - p(m_i \mid z_{1:t-1}, x_{1:t-1})} \frac{1 - p(m_i)}{p(m_i)} \end{aligned}$$

Static State Binary Bayes Filter

- By computing the ratio of both probabilities, we obtain:

$$\begin{aligned} & \frac{p(m_i \mid z_{1:t}, x_{1:t})}{1 - p(m_i \mid z_{1:t}, x_{1:t})} \\ &= \frac{p(m_i \mid z_t, x_t) p(m_i \mid z_{1:t-1}, x_{1:t-1}) p(\neg m_i)}{p(\neg m_i \mid z_t, x_t) p(\neg m_i \mid z_{1:t-1}, x_{1:t-1}) p(m_i)} \\ &= \underbrace{\frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)}}_{\text{uses } z_t} \underbrace{\frac{p(m_i \mid z_{1:t-1}, x_{1:t-1})}{1 - p(m_i \mid z_{1:t-1}, x_{1:t-1})}}_{\text{recursive term}} \underbrace{\frac{1 - p(m_i)}{p(m_i)}}_{\text{prior}} \end{aligned}$$

From Ratio to Probability

- We can easily turn the ration into the probability

$$\frac{p(x)}{1 - p(x)} = Y$$

$$p(x) = Y - Y p(x)$$

$$p(x) (1 + Y) = Y$$

$$p(x) = \frac{Y}{1 + Y}$$

$$p(x) = \frac{1}{1 + \frac{1}{Y}}$$

From Ratio to Probability

- Using $p(x) = [1 + Y^{-1}]^{-1}$ directly leads to

$$\begin{aligned} p(m_i \mid z_{1:t}, x_{1:t}) \\ = \left[1 + \frac{1 - p(m_i \mid z_t, x_t)}{p(m_i \mid z_t, x_t)} \frac{1 - p(m_i \mid z_{1:t-1}, x_{1:t-1})}{p(m_i \mid z_{1:t-1}, x_{1:t-1})} \frac{p(m_i)}{1 - p(m_i)} \right]^{-1} \end{aligned}$$

For reasons of efficiency, one performs the calculations in the log odds notation

Log Odds Notation

- The log odds notation computes the logarithm of the ratio of probabilities

$$\frac{p(m_i \mid z_{1:t}, x_{1:t})}{1 - p(m_i \mid z_{1:t}, x_{1:t})} = \underbrace{\frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)}}_{\text{uses } z_t} \underbrace{\frac{p(m_i \mid z_{1:t-1}, x_{1:t-1})}{1 - p(m_i \mid z_{1:t-1}, x_{1:t-1})}}_{\text{recursive term}} \underbrace{\frac{1 - p(m_i)}{p(m_i)}}_{\text{prior}}$$

$$\Rightarrow l(m_i \mid z_{1:t}, x_{1:t}) = \log \left(\frac{p(m_i \mid z_{1:t}, x_{1:t})}{1 - p(m_i \mid z_{1:t}, x_{1:t})} \right)$$

Log Odds Notation

- Log odds ratio is defined as

$$l(x) = \log \frac{p(x)}{1 - p(x)}$$

- and with the ability to retrieve $p(x)$

$$p(x) = \frac{\exp l(x)}{1 + \exp l(x)}$$

Occupancy Mapping in Log Odds Form

- The product turns into a sum

$$\begin{aligned} l(m_i \mid z_{1:t}, x_{1:t}) \\ = \underbrace{l(m_i \mid z_t, x_t)}_{\text{inverse sensor model}} + \underbrace{l(m_i \mid z_{1:t-1}, x_{1:t-1})}_{\text{recursive term}} - \underbrace{l(m_i)}_{\text{prior}} \end{aligned}$$

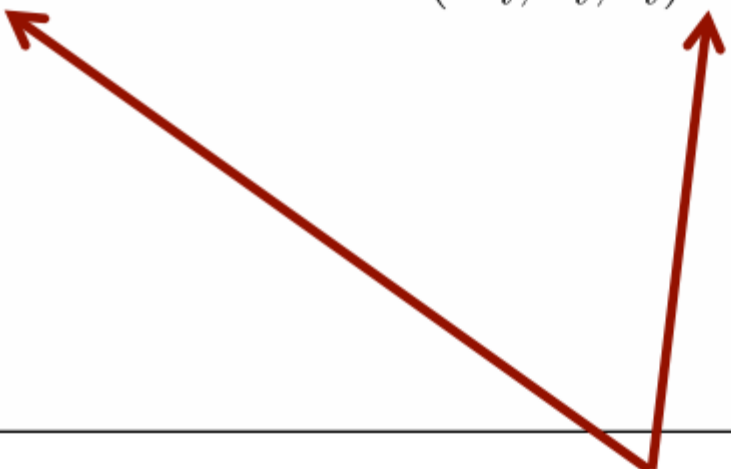
- or in short

$$l_{t,i} = \text{inv_sensor_model}(m_i, x_t, z_t) + l_{t-1,i} - l_0$$

Occupancy Mapping Algorithm

occupancy_grid_mapping($\{l_{t-1,i}\}, x_t, z_t$):

```
1:   for all cells  $m_i$  do
2:     if  $m_i$  in perceptual field of  $z_t$  then
3:        $l_{t,i} = l_{t-1,i} + \text{inv\_sensor\_model}(m_i, x_t, z_t) - l_0$ 
4:     else
5:        $l_{t,i} = l_{t-1,i}$ 
6:     endif
7:   endfor
8:   return  $\{l_{t,i}\}$ 
```

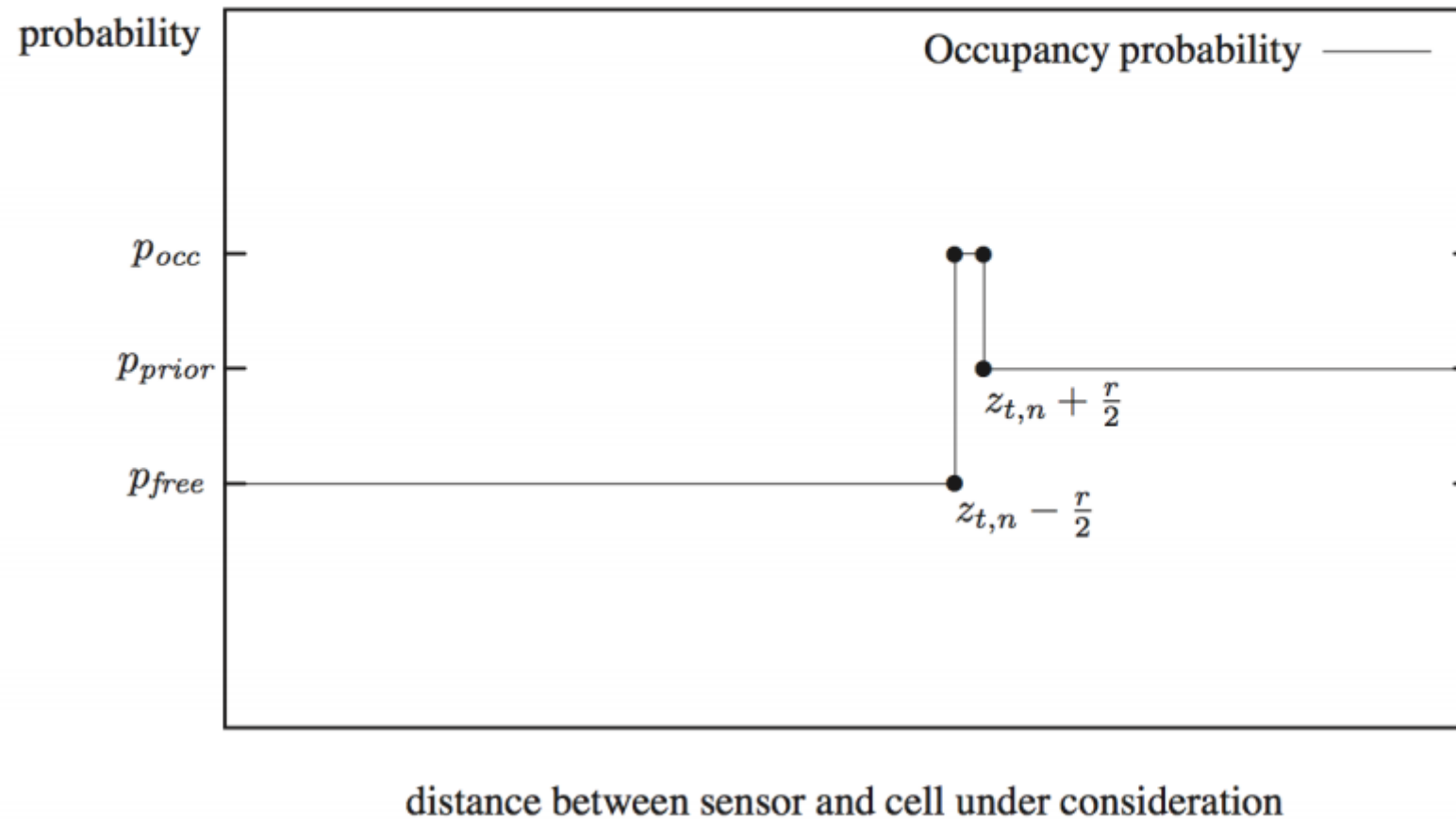


highly efficient, we only have to compute sums

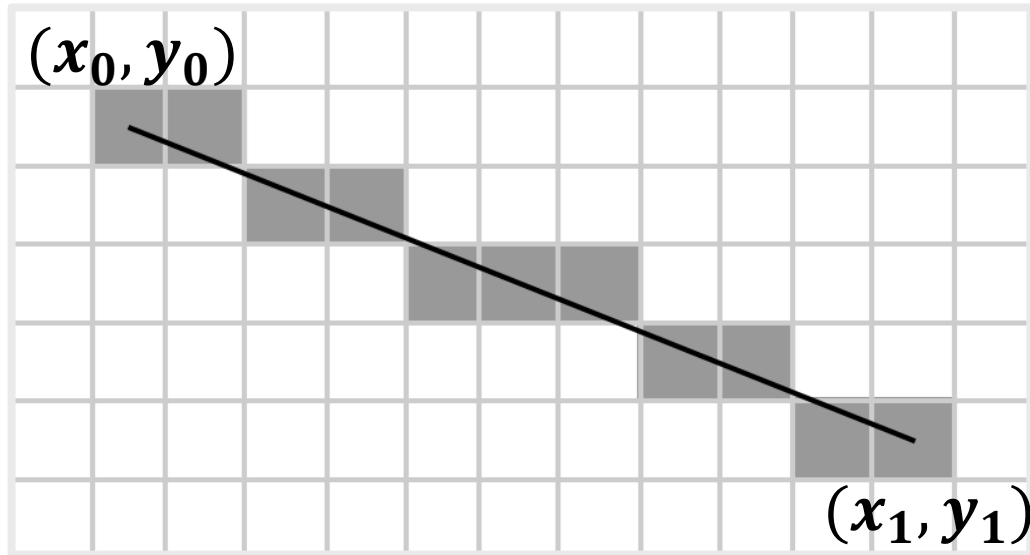
Occupancy Grid Mapping

- Moravec and Elfes proposed occupancy grid mapping in the mid 80'ies
- Developed for noisy sonar sensors
- Also called "mapping with know poses"

Inverse Sensor Model for Laser Range Finders



Bresenham's line Algorithm



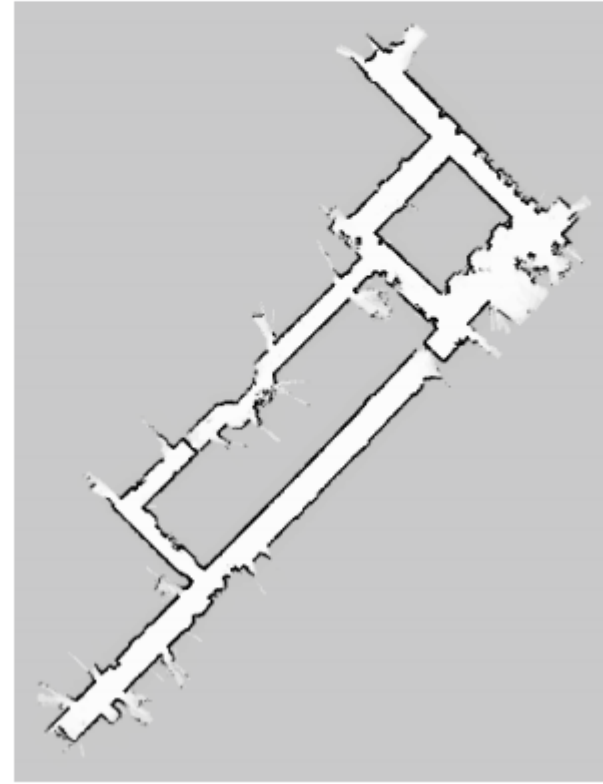
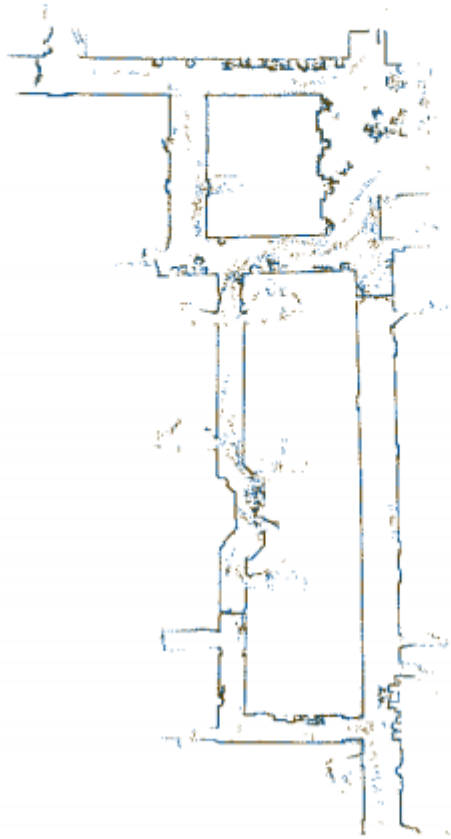
$$y - y_0 = \frac{y_1 - y_0}{x_1 - x_0} (x - x_0)$$

$$\frac{y_1 - y_0}{x_1 - x_0} (x - x_0) + y_0$$

Bresenham's line algorithm is named after Jack Elton Bresenham who developed it in 1962

```
function line(x0, x1, y0, y1)
  int deltax := x1 - x0
  int deltay := y1 - y0
  real error := 0
  real deltaerr := deltay / deltax
  int y := y0
  for x from x0 to x1
    plot(x,y)
    error := error + deltaerr
    if abs (error) ≥ 0.5 then
      y := y + 1
      error := error - 1.0
```

Occupancy Grids From Laser Scans to Maps



Occupancy Grid Map Summary

- Occupancy grid maps discretize the space into independent cells
- Each cell is a binary random variable estimating if the cell is occupied
- Static state binary Bayes filter per cell
- Mapping with known poses is easy
- Log odds model is fast to compute
- No need for predefined features

Thank you for your attention